

Natural Language Processing and Text Processing (NLTP)

Complete Study Material – Deep Research Enhanced Edition

Generated from course material with all numericals solved
Enhanced with additional diagrams, theoretical analysis, and worked examples

May 11, 2026

Contents

1	Module 1: N-gram Language Models and Edit Distance	2
1.1	Introduction to Language Models	2
1.2	N-gram Language Models	2
1.2.1	Chain Rule of Probability	3
1.2.2	The Markov Assumption	3
1.2.3	Maximum Likelihood Estimation (MLE)	4
1.2.4	Log Probabilities	5
1.3	Evaluating Language Models	5
1.3.1	Training, Test, and Development Sets	6
1.3.2	Extrinsic vs. Intrinsic Evaluation	6
1.3.3	Perplexity	6
1.4	Smoothing, Interpolation, and Backoff	9
1.4.1	The Zero-Probability Problem	9
1.4.2	Laplace (Add-One) Smoothing	9
1.4.3	Add-k Smoothing	11
1.4.4	Language Model Interpolation	11
1.4.5	Stupid Backoff	11
1.5	Entropy and Cross-Entropy	11
1.6	Edit Distance	13
2	Module 2: Word Senses, WordNet, and Vector Semantics	16
2.1	Word Senses and Polysemy	16
2.2	Relations Between Senses	17
2.2.1	Synonymy	17
2.2.2	Antonymy	17
2.2.3	Taxonomic Relations: Hyponymy and Hypernymy	17
2.2.4	Meronymy	17
2.3	WordNet	18
2.3.1	Sense Relations in WordNet	18
2.4	Word Sense Disambiguation (WSD)	19
2.4.1	WSD Algorithms	19
2.5	Pointwise Mutual Information (PMI)	20
2.5.1	Cosine Similarity in Vector Spaces	23
2.6	Word Sense Induction (WSI)	24

3	Module 3: Naive Bayes and Text Classification	26
3.1	Introduction to Text Classification	26
3.2	Naive Bayes Classifiers	26
3.2.1	Bayes' Rule and the Classification Decision	26
3.2.2	The Bag of Words Assumption	27
3.2.3	The Naive Bayes Assumption: Conditional Independence	27
3.2.4	Training the Naive Bayes Classifier	28
3.3	Optimizations for Sentiment Analysis	31
3.3.1	Binary Multinomial Naive Bayes	31
3.3.2	Negation Handling	32
3.3.3	Sentiment Lexicons	32
4	Module 4: Deep Learning Architectures for Sequence Processing	33
4.1	Recurrent Neural Networks (RNNs)	33
4.1.1	RNN Architecture	33
4.1.2	Training RNNs: Backpropagation Through Time (BPTT)	34
4.1.3	RNN as Language Model	34
4.1.4	Autoregressive Generation	35
4.2	Sequence Labeling and Sequence Classification	35
4.3	Vanishing and Exploding Gradients	35
4.4	Long Short-Term Memory (LSTM)	36
4.4.1	The Forget Gate	36
4.4.2	The Input Gate	36
4.4.3	The Output Gate	37
4.5	Gated Recurrent Unit (GRU)	37
4.5.1	The Reset Gate	37
4.5.2	The Update Gate	37
4.6	Bidirectional Recurrent Neural Networks (Bi-RNN)	42
4.6.1	Advantages and Disadvantages of Bi-RNNs	42
5	Module 5: Morphology and Syntax	44
5.1	Morphology	44
5.1.1	Morphemes	45
5.1.2	Types of Morphemes	45
5.1.3	Lexical vs. Grammatical Morphemes	46
5.1.4	Inflectional vs. Derivational Morphology	46
5.2	Regular Expressions	47
5.2.1	Disjunction	47
5.2.2	Kleene Operators	47
5.2.3	Anchors	48
5.2.4	Capture Groups and Backreferences	48
5.2.5	Errors in Pattern Matching	48
5.3	Finite State Automata	48
5.3.1	Deterministic Finite Automata (DFA)	49
5.4	Morphological Parsing	49
5.4.1	Finite State Transducers (FST)	50
5.4.2	Two-Level Morphology	50
5.5	Stemming and Lemmatization	51
5.6	The Porter Stemmer Algorithm	51
5.6.1	Measure of a Word	51
5.6.2	Porter Stemmer Steps	51
5.7	Part-of-Speech (POS) Tagging	53
5.7.1	POS Tag Sets	53

5.7.2	POS Tagging Approaches	53
5.8	Hidden Markov Model (HMM) for POS Tagging	54
5.8.1	HMM Components	54
5.8.2	HMM Decoding	55
5.9	The Viterbi Algorithm	55
5.10	Context-Free Grammars (CFG)	58
5.10.1	Example CFG Rules	59
5.10.2	Syntactic Ambiguity	59
5.11	CKY Parsing Algorithm	60
5.12	Probabilistic Context-Free Grammar (PCFG)	60
5.13	Dependency Parsing	61
5.13.1	Key Terminology	61
5.13.2	Properties of Dependency Trees	62
5.13.3	Projectivity	62
5.13.4	Shift-Reduce Parsing (Arc Standard)	62
6	Module 6: Discourse Analysis	74
6.1	Introduction to Discourse Analysis	74
6.2	Coreference Resolution	74
6.2.1	The Discourse Model	74
6.2.2	References and Anaphora	74
6.2.3	Anaphora vs. Coreference	75
6.2.4	Cataphora	75
6.2.5	Types of Referring Expressions	75
6.2.6	Types of Referents	76
6.2.7	Coreference Resolution Tasks	76
6.3	Constraints and Preferences in Coreference Resolution	76
6.3.1	Syntactic and Semantic Constraints	77
6.3.2	Preferences in Pronoun Interpretation	77
6.3.3	Hobbs Algorithm	77
6.4	Coherence Relations	79
6.5	Discourse Structure Parsing	80
6.6	Global Coherence	80
7	Summary of Key Concepts by Module	81
7.1	Module 1: N-gram Language Models and Edit Distance	81
7.2	Module 2: Word Senses, WordNet, and Vector Semantics	82
7.3	Module 3: Naive Bayes and Text Classification	82
7.4	Module 4: Deep Learning for Sequences	82
7.5	Module 5: Morphology and Syntax	82
7.6	Module 6: Discourse Analysis	83

1 Module 1: N-gram Language Models and Edit Distance

1.1 Introduction to Language Models

A **language model** is a fundamental component of many natural language processing systems. In its most basic form, a language model is a machine learning model that predicts upcoming words in a sequence. More formally, a language model assigns a probability to each possible next word, or equivalently, it defines a probability distribution over the entire vocabulary of possible next words given some preceding context. This probabilistic framework allows language models to quantify how likely a particular sequence of words is to occur in a language, which makes them useful for a wide range of applications including speech recognition, machine translation, spelling correction, and text generation.

The power of a language model lies in its ability to capture the statistical regularities of a language. For example, given the partial sentence “The cat sat on the___”, a good language model should assign a much higher probability to “mat” than to “refrigerator”, because the former is far more likely in natural English. Language models can also assign a probability to an entire sentence, which is useful for ranking candidate outputs in tasks like machine translation or speech recognition, where multiple candidate sentences might be generated and the one with the highest probability according to the language model is preferred.

Applications of language models are extensive and form the backbone of many practical NLP systems:

- **Spelling and grammar correction:** A language model can detect errors such as “Their are” (which should be “There are”) by assigning a much lower probability to the incorrect sequence.
- **Speech recognition disambiguation:** When a speech recognizer is uncertain between acoustically similar words (e.g., “recognize speech” vs. “wreck a nice beach”), a language model helps choose the more probable sequence.
- **Augmentative and Alternative Communication (AAC):** Language models can predict the next word for users with disabilities, reducing the effort required to type or communicate.
- **Machine translation:** Language models score candidate translations to select the most fluent output.

1.2 N-gram Language Models

An **n-gram** is a contiguous sequence of n items from a given text or speech. In the context of language modeling, these items are typically words, though they can also be characters or phonemes. The simplest n-grams are:

- **Unigram** (1-gram): A single word, such as “The”.
- **Bigram** (2-gram): A two-word sequence, such as “The water”.
- **Trigram** (3-gram): A three-word sequence, such as “The water of”.

An n-gram language model estimates the probability of a word based on the previous $n - 1$ words. The most common types are unigram, bigram, and trigram models, though higher-order models (4-grams, 5-grams) are also used when sufficient training data is available. The choice of n involves a trade-off: higher-order models capture more context and generally make better predictions, but they require exponentially more data to estimate reliably and become increasingly sparse (many possible n-grams never appear in the training data).

1.2.1 Chain Rule of Probability

The foundation of n-gram language modeling is the **chain rule of probability** (also called the product rule). The chain rule allows us to decompose the joint probability of a sequence of words into a product of conditional probabilities. For a sequence of words $w_1, w_2, \dots, w_n$, the joint probability is:

$$P(w_1, w_2, \dots, w_n) = P(w_1) \times P(w_2 | w_1) \times P(w_3 | w_1, w_2) \times \dots \times P(w_n | w_1, w_2, \dots, w_{n-1})$$

This equation is exact, but computing the conditional probability $P(w_n | w_1, \dots, w_{n-1})$ requires knowing the probability of a word given its entire history, which is impractical because the number of possible histories grows exponentially with sentence length. This motivates the Markov assumption.

Chain Rule Decomposition

$$\begin{array}{c} | \\ P(w_1) \\ | \\ P(w_2 | w_1) \\ | \\ P(w_3 | w_1, w_2) \\ | \\ \dots \end{array}$$

Worked Example: Chain Rule Step-by-Step

Given: Corpus = ["the cat sat", "the dog ran", "the cat ate"]

Calculate: $P(\text{the cat sat})$ using the chain rule.

Solution:

$$P(\text{the cat sat}) = P(\text{the}) \times P(\text{cat} | \text{the}) \times P(\text{sat} | \text{the cat})$$

1. $P(\text{the}) = \frac{3}{9} = 0.333$

2. $P(\text{cat} | \text{the}) = \frac{2}{3} = 0.667$

3. $P(\text{sat} | \text{the cat}) = \frac{1}{2} = 0.5$

4. $P(\text{the cat sat}) = 0.333 \times 0.667 \times 0.5 = 0.111$

Using bigram approximation (Markov assumption):

$$P(\text{the cat sat}) \approx P(\text{the}) \times P(\text{cat} | \text{the}) \times P(\text{sat} | \text{cat})$$

$$P(\text{sat} | \text{cat}) = \frac{1}{2} = 0.5, \text{ giving same result here but differs in general.}$$

1.2.2 The Markov Assumption

The **Markov assumption** simplifies the chain rule by assuming that the probability of a word depends only on a finite window of preceding words, rather than the entire history. This is also called the **Markov property** or **limited horizon** assumption. Formally, for a bigram model, the probability of a word depends only on the immediately preceding word:

$$P(w_n | w_1, w_2, \dots, w_{n-1}) \approx P(w_n | w_{n-1})$$

For a general n-gram model, the probability depends on the previous $N - 1$ words:

$$P(w_n \mid w_1, w_2, \dots, w_{n-1}) \approx P(w_n \mid w_{n-N+1}, \dots, w_{n-1})$$

This assumption is clearly false in a strict sense—the meaning of a word often depends on words much earlier in the sentence. However, the Markov assumption makes language models computationally tractable and, in practice, bigram and trigram models capture enough local context to be surprisingly effective for many tasks.

1.2.3 Maximum Likelihood Estimation (MLE)

To estimate the conditional probabilities in an n-gram model, we use **Maximum Likelihood Estimation (MLE)**, which simply counts how often each n-gram occurs in a training corpus and normalizes these counts. For a bigram model, the MLE estimate is:

$$P(w_n \mid w_{n-1}) = \frac{C(w_{n-1}, w_n)}{C(w_{n-1})}$$

where $C(w_{n-1}, w_n)$ is the count of the bigram (the number of times the two-word sequence appears together in the training corpus), and $C(w_{n-1})$ is the count of the first word (the number of times the context word appears).

For a general n-gram, the MLE estimate is:

$$P(w_n \mid w_{n-N+1}, \dots, w_{n-1}) = \frac{C(w_{n-N+1}, \dots, w_n)}{C(w_{n-N+1}, \dots, w_{n-1})}$$

This estimation method is straightforward but has a critical limitation: if a particular n-gram never appears in the training data, its MLE probability is zero, which can cause the entire sentence probability to collapse to zero.

Worked Example 1: Berkeley Restaurant Project Bigram Counts

Consider a mini-corpus with sentences augmented by start-of-sentence $\langle s \rangle$ and end-of-sentence $\langle /s \rangle$ markers:

- $\langle s \rangle$ I am Sam $\langle /s \rangle$
- $\langle s \rangle$ Sam I am $\langle /s \rangle$
- $\langle s \rangle$ I do not like green eggs and ham $\langle /s \rangle$

Sample bigram probability calculations using MLE:

$$P(I \mid \langle s \rangle) = \frac{2}{3} = 0.67, \quad P(\text{Sam} \mid \langle s \rangle) = \frac{1}{3} = 0.33$$

$$P(\text{am} \mid I) = \frac{2}{3} = 0.67, \quad P(\text{do} \mid I) = \frac{1}{3} = 0.33$$

$$P(\langle /s \rangle \mid \text{Sam}) = \frac{1}{2} = 0.5, \quad P(\text{Sam} \mid \text{am}) = \frac{1}{2} = 0.5$$

The sentence probability is the product of all bigram probabilities along the path through the sentence. For example:

$$P(\langle s \rangle \text{ I want english food } \langle /s \rangle) = P(I \mid \langle s \rangle) \times P(\text{want} \mid I) \times P(\text{english} \mid \text{want}) \times P(\text{food} \mid \text{english}) \times P(\langle /s \rangle \mid \text{food})$$

Worked Example 2: Trigram Probability Calculation

Given: Corpus = ["the cat sat", "the cat ate", "the dog ran"]

Calculate: $P(\text{sat} \mid \text{the cat})$ using trigram MLE

Solution:

1. **Count trigrams:** $C(\text{the cat sat}) = 1$, $C(\text{the cat ate}) = 1$
2. **Count bigram context:** $C(\text{the cat}) = 2$
3. **Trigram probability:** $P(\text{sat} \mid \text{the cat}) = \frac{1}{2} = 0.5$
4. $P(\text{ate} \mid \text{the cat}) = \frac{1}{2} = 0.5$

Calculate sentence probability for "the cat sat":

$$P(\text{the cat sat}) = P(\text{the}) \times P(\text{cat} \mid \text{the}) \times P(\text{sat} \mid \text{the cat})$$

$$P(\text{the}) = \frac{3}{9} = 0.333, \quad P(\text{cat} \mid \text{the}) = \frac{2}{3} = 0.667$$

$$P(\text{the cat sat}) = 0.333 \times 0.667 \times 0.5 = 0.111$$

Worked Example 3: Bigram Probability with Add-One Smoothing

Given: Mini-corpus = ["a b a", "a b b"], $V = \{a, b, \langle s \rangle, \langle /s \rangle\} = 4$

Calculate: $P(a \mid \langle s \rangle)$ with Laplace smoothing

Solution:

1. **Original counts:** $C(\langle s \rangle) = 2$, $C(\langle s \rangle, a) = 2$
2. **Original MLE:** $P(a \mid \langle s \rangle) = \frac{2}{2} = 1.0$
3. **Laplace smoothed:** $P(a \mid \langle s \rangle) = \frac{2+1}{2+4} = \frac{3}{6} = 0.5$
4. **Unseen bigram** $P(c \mid \langle s \rangle) = \frac{0+1}{2+4} = \frac{1}{6} = 0.167$

Observation: The probability mass of 0.5 is redistributed to unseen bigrams.

1.2.4 Log Probabilities

Language model probabilities are typically stored and computed as **log probabilities** to avoid numerical underflow. When we multiply many small probabilities together (as we do when computing the probability of a long sentence), the product can become smaller than the smallest representable floating-point number, causing underflow to zero. By working in log space, multiplication becomes addition, which is both numerically stable and computationally faster:

$$\log(p_1 \times p_2 \times p_3 \times p_4) = \log p_1 + \log p_2 + \log p_3 + \log p_4$$

Since log is a monotonically increasing function, the word sequence with the highest probability also has the highest log probability, so we can safely work in log space without changing the ranking of candidate sequences.

1.3 Evaluating Language Models

Before we can use a language model, we must evaluate its quality. There are two primary approaches to evaluation: extrinsic and intrinsic. Understanding the distinction is important because different evaluation methods serve different purposes and have different costs.

1.3.1 Training, Test, and Development Sets

When building any machine learning model, we partition our data into distinct subsets to ensure unbiased evaluation:

- **Training set:** The data used to learn the model parameters. For an n-gram model, this means counting n-gram frequencies and computing probabilities.
- **Test set:** A held-out portion of the data that the model has never seen during training. This gives an unbiased estimate of how the model will perform on new, unseen text.
- **Development set** (or validation set): An intermediate set used for tuning hyperparameters, trying different model variants, and making design decisions without contaminating the test set.

Data contamination occurs when the test set is used during training or when information from the test set leaks into the model development process. This leads to artificially optimistic performance estimates and models that fail to generalize to truly new data.

1.3.2 Extrinsic vs. Intrinsic Evaluation

Extrinsic evaluation embeds the language model in an end-to-end application and measures whether the application performs better. For example, we might compare two language models by measuring which one yields lower word error rate in a speech recognition system. While extrinsic evaluation is the ultimate measure of usefulness, it is expensive and time-consuming because it requires building and running the full application for every model variant.

Intrinsic evaluation measures the quality of the model independently of any specific application. The most common intrinsic metric for language models is **perplexity**, which is fast to compute and correlates reasonably well with extrinsic performance. Intrinsic evaluation allows researchers to compare many model variants quickly before selecting the best for extrinsic testing.

1.3.3 Perplexity

Perplexity (often abbreviated as PP or PPL) is the standard intrinsic evaluation metric for language models. Intuitively, perplexity measures how “surprised” the model is by the test data. A model that assigns high probabilities to the words that actually appear in the test set has low perplexity; a model that is frequently surprised by the test data has high perplexity.

Mathematically, perplexity is the inverse probability of the test set, normalized by the number of words N :

$$\text{Perplexity}(W) = P(w_1, w_2, \dots, w_N)^{-\frac{1}{N}}$$

For a bigram model, using the chain rule, this becomes:

$$\text{Perplexity}(W) = \sqrt[N]{\prod_{i=1}^N \frac{1}{P(w_i | w_{i-1})}}$$

An important property of perplexity is that it can also be interpreted as the **weighted average branching factor** of the language. The branching factor is the number of possible next words that could follow any given word. For a language with vocabulary size V , the maximum perplexity is V (uniform distribution), and the minimum is 1 (perfect prediction).

Worked Example 4: Perplexity as Weighted Average Branching Factor

Given: A language $L = \{\text{red, blue, green}\}$ with branching factor 3.

Model A: Each word follows with equal probability $\frac{1}{3}$. Test sequence: “red red red red blue” ($N = 5$)

$$\text{Perplexity}_A = \left(\frac{1}{3}\right)^{-\frac{5}{5}} = \left(\frac{1}{3}\right)^{-1} = 3$$

Model B: $P(\text{red}) = 0.8$, $P(\text{green}) = 0.1$, $P(\text{blue}) = 0.1$

$$P(\text{red red red red blue}) = 0.8^4 \times 0.1 = 0.04096$$

$$\text{Perplexity}_B = (0.04096)^{-\frac{1}{5}} = 0.527^{-1} = 1.89$$

Interpretation: Although both models have the same branching factor (3 possible next words), Model B has lower perplexity because it predicts “red” correctly most of the time. The probabilities assigned by the model directly determine how surprised it is by the test data.

Worked Example 5: Perplexity on WSJ Corpus

The table below shows perplexity values for unigram, bigram, and trigram models trained on 38 million words from the Wall Street Journal, evaluated on a 1.5 million word test set:

Unigram	Bigram	Trigram
962	170	109

As expected, higher-order models (which incorporate more context) assign higher probabilities to the test data and therefore achieve lower perplexity. The trigram model is nearly 9 times better than the unigram model by this metric.

Worked Example 6: Perplexity of a Test Sentence

Given: Test sentence = “the cat sat on the mat” ($N = 6$ tokens) Unigram model: $P(\text{the}) = 0.3$, $P(\text{cat}) = 0.1$, $P(\text{sat}) = 0.05$, $P(\text{on}) = 0.08$, $P(\text{mat}) = 0.02$

Calculate: Perplexity of the test sentence

Solution:

1. **Sentence probability** under unigram model:

$$P(W) = 0.3 \times 0.1 \times 0.05 \times 0.08 \times 0.3 \times 0.02 = 7.2 \times 10^{-7}$$

2. **Perplexity:**

$$\text{PP}(W) = P(W)^{-1/N} = (7.2 \times 10^{-7})^{-1/6} = (7.2 \times 10^{-7})^{-0.167}$$

$$\text{PP}(W) = \frac{1}{(7.2 \times 10^{-7})^{1/6}} = \frac{1}{0.068} \approx 14.7$$

Interpretation: The model is about as confused as if it had 15 equally likely choices at each step.

Worked Example 7: Comparing Model Perplexities

Given: Test sentence = “a a a a a” ($N = 5$)

Model A: $P(a) = 0.9$, $P(b) = 0.1$ **Model B:** $P(a) = 0.5$, $P(b) = 0.5$

Calculate: Perplexity for each model

Solution:

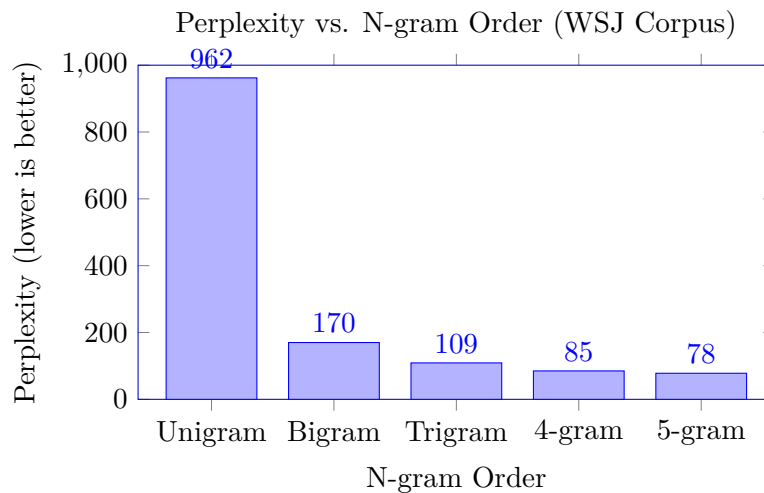
1. **Model A:** $P(W) = 0.9^5 = 0.59049$

$$PP_A = (0.59049)^{-1/5} = (0.59049)^{-0.2} = 1.19$$

2. **Model B:** $P(W) = 0.5^5 = 0.03125$

$$PP_B = (0.03125)^{-1/5} = (0.03125)^{-0.2} = 2.0$$

Conclusion: Model A has lower perplexity (1.19 vs 2.0) because it correctly predicts that “a” is the most frequent word.



Theoretical Analysis: Perplexity and Cross-Entropy

Perplexity is mathematically equivalent to the exponential of the cross-entropy:

$$\text{Perplexity}(W) = 2^{H(W)} = 2^{-\frac{1}{N} \log_2 P(w_1, w_2, \dots, w_N)}$$

where $H(W)$ is the average log probability (cross-entropy) of the test set.

Key Properties:

1. **Branching Factor Interpretation:** A perplexity of k means the model is as confused as if it had k equally likely choices at each step.
2. **Lower Bound:** Perplexity ≥ 1 (perfect prediction gives perplexity = 1).
3. **Upper Bound:** Perplexity $\leq |V|$ (uniform distribution over vocabulary).
4. **Comparison:** Perplexity can only be compared across models with the same vocabulary and test set.

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

Figure 1: Evaluating Language Models: Visual summary of perplexity concepts from Module 1 source material (Evaluating Language Models PPT).

1.4 Smoothing, Interpolation, and Backoff

A critical problem with Maximum Likelihood Estimation is that any n-gram that never appears in the finite training corpus receives a probability of zero. This is problematic for two reasons: first, it means the model will assign zero probability to any test sentence containing an unseen n-gram, making perplexity undefined (division by zero). Second, even if the test sentence does contain a zero-probability n-gram, the model is essentially declaring that this sequence is impossible, which is almost certainly wrong—there are many perfectly grammatical word sequences that simply did not happen to appear in the training data.

Smoothing (also called **discounting**) is the standard solution. Smoothing algorithms “shave off” a small amount of probability mass from the n-grams that were seen in training and redistribute that mass to the unseen n-grams, giving them small but non-zero probabilities.

1.4.1 The Zero-Probability Problem

Any finite training corpus will miss some perfectly acceptable word sequences. These missing sequences are called **zeros**. For example, a training corpus might contain “ruby” and “slippers” individually but never the phrase “ruby slippers.” If we encounter “ruby slippers” in the test data, a bigram model without smoothing would compute:

$$P(\text{slippers} \mid \text{ruby}) = \frac{C(\text{ruby}, \text{slippers})}{C(\text{ruby})} = \frac{0}{C(\text{ruby})} = 0$$

If any word in a sentence has zero probability, the entire sentence probability becomes zero, and we cannot compute perplexity at all because we would need to divide by zero.

1.4.2 Laplace (Add-One) Smoothing

The simplest smoothing method is **Laplace smoothing**, also called **add-one smoothing**. The idea is to add one to the count of every n-gram before normalizing. Every n-gram that previously had a count of zero now has a count of one; every n-gram that previously had count k now has count $k + 1$.

For unigrams:

$$P_{\text{Laplace}}(w_i) = \frac{c_i + 1}{N + V}$$

where c_i is the count of word w_i , N is the total number of word tokens, and V is the vocabulary size. The denominator increases by V because we added one to the count of each of the V vocabulary words.

For bigrams:

$$P_{\text{Laplace}}(w_n \mid w_{n-1}) = \frac{C(w_{n-1}, w_n) + 1}{C(w_{n-1}) + V}$$

Laplace smoothing is easy to understand and implement, but it tends to move too much probability mass to unseen events, making it impractical for modern language modeling. It remains useful, however, as a baseline and for text classification tasks where the smoothing behavior is less critical.

Worked Example 8: Laplace Smoothing on a Bigram

Given: Berkeley Restaurant Project corpus with $V = 1446$.

Original bigram count for “want to”: $C(\text{want}, \text{to}) = 608$, $C(\text{want}) = 927$.

$$P_{\text{MLE}}(\text{to} \mid \text{want}) = \frac{608}{927} = 0.66$$

After Laplace smoothing:

$$P_{\text{Laplace}}(\text{to} \mid \text{want}) = \frac{608 + 1}{927 + 1446} = \frac{609}{2373} = 0.26$$

Observation: Laplace smoothing reduces the probability of frequent bigrams dramatically (from 0.66 to 0.26) while giving unseen bigrams a small probability of $\frac{1}{2373} \approx 0.00042$. This is generally too aggressive for language modeling.

Worked Example 9: Laplace Smoothing on Unigrams

Given: Vocabulary $V = \{a, b, c, d\}$, corpus: “a a b a c b a d”

Original counts: $C(a) = 4$, $C(b) = 2$, $C(c) = 1$, $C(d) = 1$, $N = 8$

Calculate: Smoothed probabilities with Laplace (add-1)

Solution:

$$1. P_{\text{Laplace}}(a) = \frac{4+1}{8+4} = \frac{5}{12} = 0.417$$

$$2. P_{\text{Laplace}}(b) = \frac{2+1}{8+4} = \frac{3}{12} = 0.250$$

$$3. P_{\text{Laplace}}(c) = \frac{1+1}{8+4} = \frac{2}{12} = 0.167$$

$$4. P_{\text{Laplace}}(d) = \frac{1+1}{8+4} = \frac{2}{12} = 0.167$$

Check: $0.417 + 0.250 + 0.167 + 0.167 = 1.001$ (rounding). Correct.

Original MLE: $P(a) = 0.5$, $P(b) = 0.25$, $P(c) = 0.125$, $P(d) = 0.125$

Observation: Probability of frequent word “a” drops from 0.5 to 0.417; probability of rare words “c” and “d” each rises from 0.125 to 0.167.

Worked Example 10: Laplace Smoothing on Unseen Bigrams

Given: Corpus: “the cat sat on the mat the dog ran” ($V = 7$)

Original MLE: $P(\text{cat} \mid \text{the}) = \frac{2}{3} = 0.667$, $P(\text{dog} \mid \text{the}) = \frac{1}{3} = 0.333$

Calculate: Smoothed probabilities and probability of unseen bigram “the fox”

Solution:

$$1. C(\text{the}) = 3, \text{ total context count for “the”}$$

$$2. P_{\text{Laplace}}(\text{cat} \mid \text{the}) = \frac{2+1}{3+7} = \frac{3}{10} = 0.3$$

$$3. P_{\text{Laplace}}(\text{dog} \mid \text{the}) = \frac{1+1}{3+7} = \frac{2}{10} = 0.2$$

$$4. P_{\text{Laplace}}(\text{fox} \mid \text{the}) = \frac{0+1}{3+7} = \frac{1}{10} = 0.1$$

Check: $0.3 + 0.2 + 0.1 + (\text{other 4 unseen words}) = 0.3 + 0.2 + 0.1 + 4 \times 0.1 = 1.0$. Correct.

1.4.3 Add-k Smoothing

An alternative to add-one smoothing is to add a smaller fractional count k (where $0 < k < 1$), typically values like 0.5 or 0.01. This method is called **add-k smoothing**:

$$P_{\text{Add-k}}(w_n | w_{n-1}) = \frac{C(w_{n-1}, w_n) + k}{C(w_{n-1}) + kV}$$

The optimal value of k depends on the corpus and task and can be determined by optimizing on a development set. While add-k smoothing is an improvement over Laplace smoothing, it still does not perform well for language modeling in practice because it produces counts with poor variance and inappropriate discounts for different frequency ranges.

1.4.4 Language Model Interpolation

Interpolation is a more sophisticated approach that combines probabilities from n-gram models of different orders. The intuition is that if a particular trigram has never been seen, we might still have useful information from the corresponding bigram or unigram. For example, even if we have never seen the trigram “natural language processing”, we might have seen “language processing” as a bigram, which gives us some useful predictive power.

In simple linear interpolation, we combine unigram, bigram, and trigram probabilities with weights $\lambda_1, \lambda_2, \lambda_3$:

$$\hat{P}(w_n | w_{n-2}, w_{n-1}) = \lambda_1 P(w_n) + \lambda_2 P(w_n | w_{n-1}) + \lambda_3 P(w_n | w_{n-2}, w_{n-1})$$

where $\lambda_1 + \lambda_2 + \lambda_3 = 1$. The λ values are typically learned from a held-out corpus by choosing the values that maximize the likelihood of that held-out data.

1.4.5 Stupid Backoff

Backoff is another alternative to interpolation. In a backoff model, if the n-gram we need has zero counts, we “back off” to a lower-order n-gram. **Stupid backoff** is a particularly simple version that does not attempt to normalize the backed-off probabilities into a true probability distribution. It simply uses a fixed weight λ (typically 0.4) to scale the lower-order probability:

$$S(w_i | w_{i-N+1:i-1}) = \begin{cases} \frac{\text{count}(w_{i-N+1:i})}{\text{count}(w_{i-N+1:i-1})} & \text{if count} > 0 \\ \lambda \cdot S(w_i | w_{i-N+2:i-1}) & \text{otherwise} \end{cases}$$

Stupid backoff is widely used in large-scale language modeling because it is simple, fast, and performs well in practice despite not being a proper probability distribution.

1.5 Entropy and Cross-Entropy

The perplexity measure actually arises from the information-theoretic concept of **cross-entropy**, which provides a deeper theoretical foundation for understanding why perplexity has the form it does.

Entropy $H(X)$ of a random variable X measures the average amount of information (in bits) needed to encode outcomes of X :

$$H(X) = - \sum_{x \in \mathcal{X}} p(x) \log_2 p(x)$$

For a sequence of words, we can define the **entropy rate** as the per-word entropy:

$$H(L) = \lim_{n \rightarrow \infty} -\frac{1}{n} \log_2 P(w_1, w_2, \dots, w_n)$$

Cross-entropy $H(p, q)$ measures the average number of bits needed to encode outcomes from distribution p using a model q :

$$H(p, q) = \lim_{n \rightarrow \infty} -\frac{1}{n} \log_2 q(w_1, w_2, \dots, w_n)$$

The key insight is that perplexity is simply 2 raised to the power of cross-entropy:

$$\text{Perplexity}(W) = 2^{H(W)}$$

This explains why perplexity uses the inverse probability and why lower perplexity corresponds to a better model.

Worked Example 11: Entropy of a Horse Race

Given: 8 horses with the following probabilities:

Horse1	Horse2	Horse3	Horse4	Horse5	Horse6	Horse7	Horse8
$\frac{1}{2}$	$\frac{1}{4}$	$\frac{1}{8}$	$\frac{1}{16}$	$\frac{1}{64}$	$\frac{1}{64}$	$\frac{1}{64}$	$\frac{1}{64}$

$$H(X) = -\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{4} \log_2 \frac{1}{4} - \frac{1}{8} \log_2 \frac{1}{8} - \frac{1}{16} \log_2 \frac{1}{16} - 4 \times \frac{1}{64} \log_2 \frac{1}{64}$$

$$H(X) = 0.5 + 0.5 + 0.375 + 0.25 + 4 \times 0.09375 = 2 \text{ bits}$$

For 8 equally likely horses, the entropy would be $\log_2 8 = 3$ bits. The non-uniform distribution requires fewer bits to encode because some outcomes are highly predictable.

Worked Example 12: Entropy of a Biased Coin

Given: Coin with $P(\text{heads}) = 0.8$, $P(\text{tails}) = 0.2$

Calculate: Entropy $H(X)$

Solution:

$$H(X) = -P(\text{heads}) \log_2 P(\text{heads}) - P(\text{tails}) \log_2 P(\text{tails})$$

$$H(X) = -0.8 \times \log_2(0.8) - 0.2 \times \log_2(0.2)$$

$$H(X) = -0.8 \times (-0.322) - 0.2 \times (-2.322)$$

$$H(X) = 0.258 + 0.464 = 0.722 \text{ bits}$$

For a fair coin ($P = 0.5$ each): $H(X) = 1$ bit.

Interpretation: The biased coin is more predictable (lower entropy) than a fair coin.

Worked Example 13: Cross-Entropy Between True and Estimated Distributions

Given: True distribution p : $P(a) = 0.5$, $P(b) = 0.3$, $P(c) = 0.2$ Model distribution q : $P(a) = 0.4$, $P(b) = 0.4$, $P(c) = 0.2$

Calculate: Cross-entropy $H(p, q)$

Solution:

$$H(p, q) = -\sum_x p(x) \log_2 q(x)$$

$$H(p, q) = -[0.5 \times \log_2(0.4) + 0.3 \times \log_2(0.4) + 0.2 \times \log_2(0.2)]$$

$$H(p, q) = -[0.5 \times (-1.322) + 0.3 \times (-1.322) + 0.2 \times (-2.322)]$$

$$H(p, q) = 0.661 + 0.397 + 0.464 = 1.522 \text{ bits}$$

True entropy $H(p) = -[0.5 \times \log_2(0.5) + 0.3 \times \log_2(0.3) + 0.2 \times \log_2(0.2)] = 1.485$ bits.

Interpretation: The cross-entropy (1.522) is slightly higher than the true entropy (1.485) because the model q is not perfectly aligned with p .

1.6 Edit Distance

Edit distance (also known as Levenshtein distance) measures the minimum number of single-character editing operations required to transform one string into another. The three allowed operations are:

- **Insertion:** Insert a character into the string.
- **Deletion:** Delete a character from the string.
- **Substitution:** Replace one character with another.

Edit distance is computed efficiently using **dynamic programming**. We construct a matrix D where $D[i, j]$ represents the edit distance between the first i characters of the source string and the first j characters of the target string. The recurrence relation is:

$$D[i, j] = \min \begin{cases} D[i-1, j] + 1 & \text{(deletion)} \\ D[i, j-1] + 1 & \text{(insertion)} \\ D[i-1, j-1] + \text{cost}(i, j) & \text{(substitution)} \end{cases}$$

where $\text{cost}(i, j) = 0$ if the i -th source character equals the j -th target character, and 1 (or 2) otherwise. The base cases are $D[0, j] = j$ and $D[i, 0] = i$.

Worked Example 14: Edit Distance – intention to execution

Given: Source = “intention”, Target = “execution”

The minimum edit distance is 5 operations (with substitution cost = 2):

1. **Delete** “i”: intention → ntention
2. **Substitute** “n” → “e”: ntention → etention
3. **Substitute** “t” → “x”: etention → exention
4. **Insert** “u”: exention → exenution
5. **Substitute** “n” → “c”: exenution → execution

Total cost: 1 (delete) + 2 (sub) + 2 (sub) + 1 (insert) + 2 (sub) = **8** (with cost=2 for substitution) or **5** operations.

Worked Example 15: Edit Distance – kitten to sitting

Given: Source = “kitten”, Target = “sitting”

Find: Minimum edit distance (standard Levenshtein, substitution cost = 1)

Solution: The alignment and operations:

k i t t e n
s i t t i n g

Operations:

1. substitute $k \rightarrow s$ (cost 1)
2. keep i (cost 0)
3. keep t (cost 0)
4. keep t (cost 0)
5. substitute $e \rightarrow i$ (cost 1)
6. keep n (cost 0)
7. insert g (cost 1)

Minimum edit distance = 3

Worked Example 16: Edit Distance – sunday to saturday

Given: Source = “sunday”, Target = “saturday”

Find: Minimum edit distance (standard Levenshtein, all costs = 1)

Solution: The alignment:

s u n d a y
s a t u r d a y

Operations:

1. keep s (cost 0)
2. substitute u $\rightarrow$ a (cost 1)
3. insert t (cost 1)
4. substitute n $\rightarrow$ u (cost 1)
5. insert r (cost 1)
6. keep d (cost 0)
7. insert a (cost 1)
8. keep y (cost 0)

Minimum edit distance = 5

DP Matrix (key values):

	$\emptyset$	s	a	t	u	r	d	a	y
$\emptyset$	0	1	2	3	4	5	6	7	8
s	1	0	1	2	3	4	5	6	7
u	2	1	1	2	2	3	4	5	6
n	3	2	2	2	3	3	4	5	6
d	4	3	3	3	3	4	3	4	5
a	5	4	3	4	4	4	4	3	4
y	6	5	4	4	5	5	5	4	3

The minimum edit distance is found by tracing back from the matrix. For “sunday” (6 letters) vs “saturday” (8 letters):

Alignment: s u n d a y vs s a t u r d a y

Operations: substitute u $\rightarrow$ a (1), insert t (1), substitute n $\rightarrow$ u (1), insert r (1), keep d (0), insert a (1), keep y (0) = 5.

Minimum edit distance = 5

X	sub[X, Y] = Substitution of X (incorrect) for Y (correct)																									
	Y (correct)																									
	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
a	0	0	7	1	342	0	0	2	118	0	1	0	0	3	76	0	0	1	35	9	9	0	1	0	5	0
b	0	0	9	9	2	2	3	1	0	0	0	5	11	5	0	10	0	0	2	1	0	0	8	0	0	0
c	6	5	0	16	0	9	5	0	0	0	1	0	7	9	1	10	2	5	39	40	1	3	7	1	1	0
d	1	10	13	0	12	0	5	5	0	0	2	3	7	3	0	1	0	43	30	22	0	0	4	0	2	0
e	388	0	3	11	0	2	2	0	89	0	0	3	0	5	93	0	0	14	12	6	15	0	1	0	18	0
f	0	15	0	3	1	0	5	2	0	0	0	3	4	1	0	0	0	6	4	12	0	0	2	0	0	0
g	4	1	11	11	9	2	0	0	0	1	1	3	0	0	2	1	3	5	13	21	0	0	1	0	3	0
h	1	8	0	3	0	0	0	0	0	0	2	0	12	14	2	3	0	3	1	11	0	0	2	0	0	0
i	103	0	0	0	146	0	1	0	0	0	0	6	0	0	49	0	0	0	2	1	47	0	2	1	15	0
j	0	1	1	9	0	0	1	0	0	0	0	2	1	0	0	0	0	5	0	0	0	0	0	0	0	0
k	1	2	8	4	1	1	2	5	0	0	0	0	5	0	2	0	0	0	6	0	0	0	4	0	0	3
l	2	10	1	4	0	4	5	6	13	0	1	0	0	14	2	5	0	11	10	2	0	0	0	0	0	0
m	1	3	7	8	0	2	0	6	0	0	4	4	0	180	0	6	0	0	9	15	13	3	2	2	3	0
n	2	7	6	5	3	0	1	19	1	0	4	35	78	0	0	7	0	28	5	7	0	0	1	2	0	2
o	91	1	1	3	116	0	0	0	25	0	2	0	0	0	0	14	0	2	4	14	39	0	0	0	18	0
p	0	11	1	2	0	6	5	0	2	9	0	2	7	6	15	0	0	1	3	6	0	4	1	0	0	0
q	0	0	1	0	0	0	27	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
r	0	14	0	30	12	2	2	8	2	0	5	8	4	20	1	14	0	0	12	22	4	0	0	1	0	0
s	11	8	27	33	35	4	0	1	0	1	0	27	0	6	1	7	0	14	0	15	0	0	5	3	20	1
t	3	4	9	42	7	5	19	5	0	1	0	14	9	5	5	6	0	11	37	0	0	2	19	0	7	6
u	20	0	0	0	44	0	0	0	64	0	0	0	0	2	43	0	0	4	0	0	0	0	2	0	8	0
v	0	0	7	0	0	3	0	0	0	0	0	1	0	0	1	0	0	0	8	3	0	0	0	0	0	0
w	2	2	1	0	1	0	0	2	0	0	1	0	0	0	0	7	0	6	3	3	1	0	0	0	0	0
x	0	0	0	2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	9	0	0	0	0	0	0	0
y	0	0	2	0	15	0	1	7	15	0	0	0	2	0	6	1	0	7	36	8	5	0	0	1	0	0
z	0	0	0	7	0	0	0	0	0	0	0	7	5	0	0	0	0	2	21	3	0	0	0	0	3	0

Figure 2: Edit Distance visualization from source material (EditDistance.pptx), showing the dynamic programming matrix and alignment path between two strings.

2 Module 2: Word Senses, WordNet, and Vector Semantics

2.1 Word Senses and Polysemy

When we encounter a word in natural language, we rarely think about the fact that most words carry multiple meanings. The study of how words relate to meaning begins with understanding that a **word sense** is a discrete representation of one aspect of the meaning of a word. For example, the word “bank” can refer to a financial institution (“I deposited money at the bank”) or the edge of a river (“We sat on the bank of the river”). Each of these is a distinct sense of the word “bank.”

The phenomenon of a single word having multiple related meanings is called **polysemy**. Polysemy is different from **homonymy**, where two words happen to share the same spelling and pronunciation but have completely unrelated meanings (e.g., “bark” as the sound a dog makes vs. “bark” as the outer covering of a tree). In polysemy, the different senses of a word are usually related in some way. For instance, the financial “bank” and the river “bank” both relate to the concept of an edge or boundary—financial institutions were historically located on the edges of marketplaces.

Structured polysemy refers to the systematic relationships that exist between the senses of a polysemous word. For example, the word “newspaper” can refer to the physical object (“The newspaper fell off the table”) or the organization that produces it (“The newspaper published an editorial”). The relationship between these two senses is a regular metonymic pattern (ORGANIZATION ↔ PHYSICAL ARTIFACT) that applies to many words.

Word senses can be distinguished by several criteria:

- **Independent truth conditions:** Two senses are distinct if a sentence can be simultaneously true under one sense and false under another. For example, “The newspaper is on the table” is true if “newspaper” means the physical object, but false if it means the organization.
- **Different syntactic behavior:** Distinct senses may have different grammatical properties. For example, “book” as a noun takes plural form (“books”), while “book” as a verb takes past tense (“booked”).

- **Independent sense relations:** Each sense may participate in different lexical relations. For example, “bank” (financial institution) is a hyponym of “financial institution”, while “bank” (river edge) is a hyponym of “shore”.
- **Antagonistic meanings (zeugma test):** If a sentence uses a word in two incompatible senses simultaneously, the result is anomalous. For example, “?He withdrew the money from the bank and sat on it” is zeugmatic because “bank” is being used as both a financial institution and a river edge.

2.2 Relations Between Senses

Lexical semantics is concerned with the relationships that hold between the meanings of words. Understanding these relationships is essential for tasks ranging from information retrieval to machine translation to question answering.

2.2.1 Synonymy

Two word senses are **synonyms** if they are identical or nearly identical in meaning, such that they can be substituted for each other in many (though not necessarily all) contexts without changing the truth conditions of the sentence. Examples include “couch” and “sofa”, “vomit” and “throw up”, and “car” and “automobile”.

It is important to note that synonymy is a relationship between *senses*, not between words. Two words may be synonyms in some senses but not in others. For example, “big” and “large” are synonyms when describing physical size, but only “large” can describe the scope of an investigation (“a large investigation” sounds natural, while “?a big investigation” is slightly less formal).

True absolute synonymy is rare because words that are very close in meaning usually differ in register (formal vs. informal), dialect, or connotation. Near-synonyms like “ask” and “request” or “error” and “mistake” illustrate this subtlety.

2.2.2 Antonymy

Antonyms are words with opposite meanings. Unlike synonymy, antonymy is a relationship between words rather than senses, because a word typically has only one antonym for each sense. Common examples include “long”/“short”, “big”/“little”, “fast”/“slow”, “cold”/“hot”, and “rise”/“fall”.

A special class of antonyms called **reversives** describes changes or movements in opposite directions. Examples include “rise”/“fall”, “up”/“down”, and “advance”/“retreat”. These are particularly important in temporal and spatial descriptions.

2.2.3 Taxonomic Relations: Hyponymy and Hypernymy

Hyponymy and **hypernymy** describe the relationship between a more specific concept and a more general concept. A **hyponym** is a word that denotes a subclass of another word. For example, “car” is a hyponym of “vehicle” because every car is a vehicle. A **hypernym** (also called a **superordinate**) is the more general word; “vehicle” is a hypernym of “car”.

The relationship is often expressed as an **IS-A hierarchy**: “A car IS-A vehicle.” This hierarchical structure is fundamental to organizing lexical knowledge because it allows inference: if we know something is a car, we automatically know it is also a vehicle and inherits all the properties of vehicles.

2.2.4 Meronymy

Meronymy is the part-whole relationship. A **meronym** is a part of something. For example, a “wheel” is a meronym of “car”, and a “leg” is a meronym of “chair”. Conversely, the whole is called a **holonym**: “car” is a holonym of “wheel”. Meronymy is important for understanding

physical descriptions and for tasks like question answering (e.g., “How many wheels does a car have?” requires knowing the meronymy relationship).

2.3 WordNet

WordNet is a large, freely available lexical database of English (and other languages) that organizes words into synonym sets called **synsets**. Rather than listing words alphabetically like a traditional dictionary, WordNet groups words into synsets based on their meanings and defines relationships between these synsets.

The English WordNet consists of three separate databases:

- **Nouns:** 117,798 entries in WordNet 3.0
- **Verbs:** 11,529 entries
- **Adjectives and adverbs:** 22,479 adjectives and 4,481 adverbs

Each synset in WordNet represents a single underlying lexical concept and is associated with a **gloss** (a short textual definition) and often example sentences. For example, the synset for the noun sense of “bank” meaning a financial institution might have the gloss: “a financial institution that accepts deposits and channels the money into lending activities.”

WordNet also organizes concepts into **supersenses**, which are lexicographic categories that provide a coarse-grained semantic classification. There are 26 supersenses for nouns (e.g., ANIMAL, ARTIFACT, FOOD, PERSON) and 15 for verbs (e.g., COMMUNICATION, COMPETITION, CONSUMPTION).

2.3.1 Sense Relations in WordNet

WordNet encodes many of the lexical relations discussed above at the sense level. The following table summarizes the major relations:

Relation	Also Called	Example
Hypernym	Superordinate	breakfast ¹ → meal ¹
Hyponym	Subordinate	meal ¹ → lunch ¹
Instance Hypernym	Instance	Austen ¹ → author ¹
Part Meronym	Has-Part	table ² → leg ³
Antonym	Semantic opposition	leader ¹ ⇔ follower ¹
Derivation	Same morphological root	destruction ¹ ⇔ destroy ¹

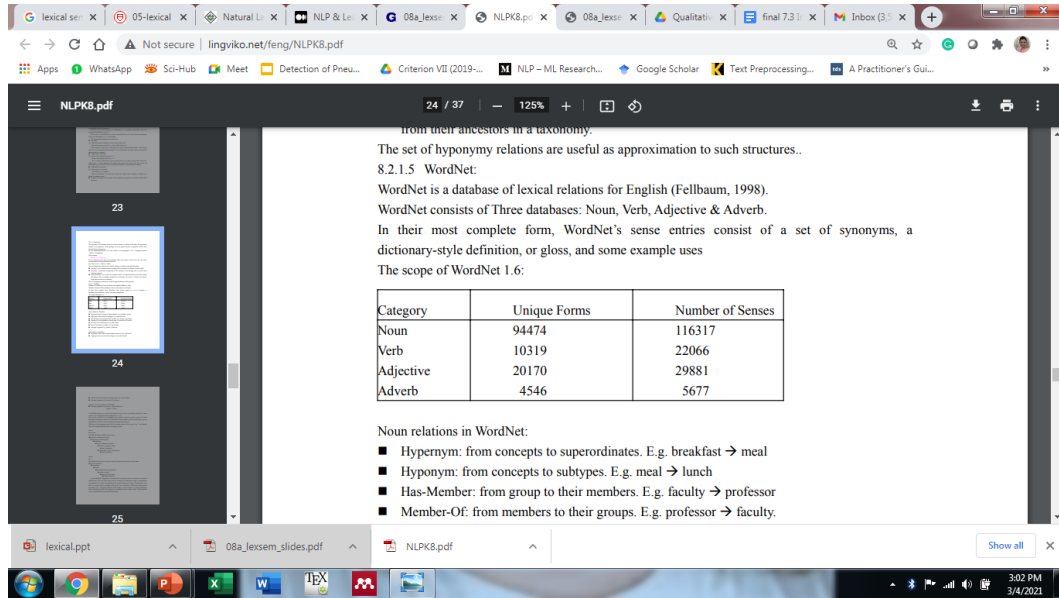


Figure 3: Word Sense and Semantic Relations visualization from source material (Word Sense Disambiguation PPT), showing the structure of sense relations and disambiguation concepts.

2.4 Word Sense Disambiguation (WSD)

Word Sense Disambiguation (WSD) is the computational task of determining which sense of a word is being used in a particular context. Because most common words are polysemous, WSD is essential for many NLP applications that require understanding the precise meaning of text.

WSD tasks come in two main varieties:

- **Lexical sample task:** A small, pre-selected set of target words is chosen, and the system must disambiguate only those words. Because the inventory of senses is small, this task is more tractable and simple supervised classification approaches work well.
- **All-words task:** The system must disambiguate every content word in a text. This is much harder because the set of possible senses is much larger, leading to data sparsity problems.

2.4.1 WSD Algorithms

Several approaches have been developed for WSD, ranging from simple baselines to sophisticated neural methods:

1. **Most Frequent Sense Baseline:** This simple but surprisingly effective baseline always chooses the most frequent sense of a word from a labeled corpus. In WordNet, senses are ordered by frequency, so this corresponds to choosing the first sense. This baseline achieves roughly 60-70% accuracy, which is difficult to beat for many words.
2. **One Sense Per Discourse:** This heuristic, based on the observation that a word appearing multiple times in a text or discourse often appears with the same sense, provides a useful bias toward consistent sense assignment within a document.
3. **Contextual Embeddings (State of the Art):** The best-performing WSD algorithm is a simple 1-nearest-neighbor approach using contextual word embeddings such as BERT. At training time, each labeled sentence is passed through BERT to produce a contextual embedding for the target word. At test time, the contextual embedding of the target word is compared to all sense embeddings, and the nearest neighbor is chosen as the predicted sense.
4. **Simplified Lesk Algorithm:** This knowledge-based approach chooses the sense whose dictionary gloss and example sentences share the most words with the target word's surrounding

context. For example, to disambiguate “bank” in “The bank can guarantee deposits will cover future tuition costs”, the algorithm compares the context words to the glosses of each sense and selects the one with the highest word overlap.

2.5 Pointwise Mutual Information (PMI)

Pointwise Mutual Information (PMI) is a measure of association between two events (or in NLP, between two words) that quantifies how much more they co-occur than would be expected by chance if they were independent. PMI is a fundamental tool in distributional semantics, the branch of semantics that studies word meaning based on patterns of word co-occurrence in large corpora.

The intuition behind PMI is straightforward: if two words appear together much more often than their individual frequencies would predict, they are likely semantically related. For example, “doctor” and “nurse” co-occur far more often than chance, suggesting a strong semantic association.

The formula for PMI between a target word w and a context word c is:

$$\text{PMI}(w, c) = \log_2 \frac{P(w, c)}{P(w)P(c)}$$

where $P(w, c)$ is the probability that w and c co-occur, $P(w)$ is the probability of w , and $P(c)$ is the probability of c . When w and c are independent, $P(w, c) = P(w)P(c)$, and PMI equals zero. Positive PMI values indicate positive association (they co-occur more than expected); negative PMI values indicate negative association (they co-occur less than expected).

Positive PMI (PPMI) is a variant that replaces negative PMI values with zero, since negative associations are often noisy and not semantically meaningful in practice:

$$\text{PPMI}(w, c) = \max \left(\log_2 \frac{P(w, c)}{P(w)P(c)}, 0 \right)$$

Numerical: PPMI Calculation

Given: Co-occurrence counts from a Wikipedia corpus:

	computer	data	result	pie	sugar	count(w)
cherry	2	8	9	442	25	486
strawberry	0	0	1	60	19	80
digital	1670	1683	85	5	4	3447
information	3325	3982	378	5	13	7703
count(c)	4997	5673	473	512	61	11716

Calculate PPMI(information, data):

Joint probability:

$$P(w = \text{information}, c = \text{data}) = \frac{3982}{11716} = 0.3399$$

Marginal probabilities:

$$P(w = \text{information}) = \frac{7703}{11716} = 0.6575, \quad P(c = \text{data}) = \frac{5673}{11716} = 0.4842$$

PMI:

$$\text{PMI}(\text{information}, \text{data}) = \log_2 \frac{0.3399}{0.6575 \times 0.4842} = \log_2(1.067) = 0.0944$$

Since PMI is positive, PPMI equals PMI: $\text{PPMI}(\text{information}, \text{data}) = 0.0944$.

Worked Example 17: PPMI for Another Word-Context Pair

Given: Same co-occurrence matrix as Example 16:

	computer	data	result	pie	sugar	count(w)
cherry	2	8	9	442	25	486
strawberry	0	0	1	60	19	80
digital	1670	1683	85	5	4	3447
information	3325	3982	378	5	13	7703
count(c)	4997	5673	473	512	61	11716

Calculate: PPMI(digital, data)

Solution:

1. Joint probability: $P(w = \text{digital}, c = \text{data}) = \frac{1683}{11716} = 0.1436$
2. Marginal probabilities:

$$P(w = \text{digital}) = \frac{3447}{11716} = 0.2942, \quad P(c = \text{data}) = \frac{5673}{11716} = 0.4842$$

3. PMI:

$$\text{PMI}(\text{digital}, \text{data}) = \log_2 \frac{0.1436}{0.2942 \times 0.4842} = \log_2(1.007) = 0.010$$

4. Since $\text{PMI} > 0$, $\text{PPMI} = \text{PMI} = \mathbf{0.010}$

Calculate: PPMI(cherry, computer)

1. Joint probability: $P(w = \text{cherry}, c = \text{computer}) = \frac{2}{11716} = 0.000171$
2. Marginal probabilities:

$$P(w = \text{cherry}) = \frac{486}{11716} = 0.0415, \quad P(c = \text{computer}) = \frac{4997}{11716} = 0.4267$$

3. PMI:

$$\text{PMI}(\text{cherry}, \text{computer}) = \log_2 \frac{0.000171}{0.0415 \times 0.4267} = \log_2(0.00966) = -6.69$$

4. Since $\text{PMI} < 0$, $\text{PPMI} = \mathbf{0}$

Interpretation: “digital” and “data” co-occur about as expected by chance (PMI near 0), while “cherry” and “computer” co-occur far less than expected (negative PMI, clipped to 0).

Worked Example 18: Building a Full PPMI Matrix

Given: Mini-corpus = ["the cat sat", "the dog ran", "the cat ran"]

Target word: "cat" **Context words:** "the", "sat", "ran", "dog"

Solution:

1. **Co-occurrence counts** (window size = 2, target-context pairs within window):

	the	sat	ran	dog	count(w)
cat	2	1	1	0	4

x Total word-context pairs = 12 (from 3 sentences, 4 pairs each).

2. **Calculate probabilities:**

$$P(w = \text{cat}) = \frac{4}{12} = 0.333, \quad P(c = \text{the}) = \frac{3}{12} = 0.250$$

$$P(\text{cat}, \text{the}) = \frac{2}{12} = 0.167$$

3. **PMI:**

$$\text{PMI}(\text{cat}, \text{the}) = \log_2 \frac{0.167}{0.333 \times 0.250} = \log_2(2.004) = 1.00$$

4. **PPMI:**

$$\text{PPMI}(\text{cat}, \text{the}) = \max(1.00, 0) = \mathbf{1.00}$$

For "cat" and "sat":

$$P(\text{cat}, \text{sat}) = \frac{1}{12} = 0.083, \quad P(\text{sat}) = \frac{1}{12} = 0.083$$

$$\text{PMI}(\text{cat}, \text{sat}) = \log_2 \frac{0.083}{0.333 \times 0.083} = \log_2(3.0) = 1.585$$

$$\text{PPMI}(\text{cat}, \text{sat}) = \mathbf{1.585}$$

For "cat" and "dog":

$$P(\text{cat}, \text{dog}) = 0, \quad \text{PMI} = -\infty, \quad \text{PPMI} = \mathbf{0}$$

- Raise the context probabilities to $\alpha = 0.75$:

- This helps because $P_\alpha(c) > P(c)$ for rare c

- Consider two events, $P(a) = .99$ and $P(b) = .01$

$$\bullet \quad P_\alpha(a) = \frac{.99^{.75}}{.99^{.75} + .01^{.75}} = .97 \quad P_\alpha(b) = \frac{.01^{.75}}{.99^{.75} + .01^{.75}} = .03$$

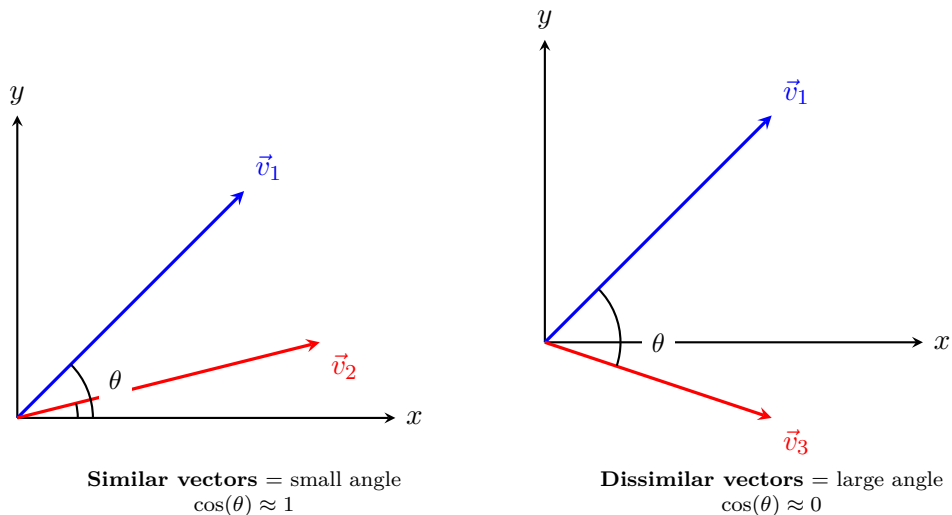
Figure 4: Vector Semantics and Distributional Hypothesis visualization from source material (Vector Semantics PPT), illustrating how word meaning is represented in vector spaces based on co-occurrence patterns.

2.5.1 Cosine Similarity in Vector Spaces

Words are represented as vectors in a high-dimensional space, where each dimension corresponds to a context word or feature. The **cosine similarity** between two word vectors measures the cosine of the angle between them:

$$\text{cosine}(\vec{u}, \vec{v}) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \|\vec{v}\|} = \frac{\sum_{i=1}^n u_i v_i}{\sqrt{\sum_{i=1}^n u_i^2} \sqrt{\sum_{i=1}^n v_i^2}}$$

Cosine similarity ranges from -1 (opposite directions) to 1 (same direction), with 0 indicating orthogonality (no similarity).



Worked Example: Cosine Similarity Computation

Given: Two word vectors in a 3-dimensional space:

$$\vec{v}_{\text{doctor}} = (5, 3, 2), \quad \vec{v}_{\text{nurse}} = (4, 4, 1)$$

Calculate: Cosine similarity between “doctor” and “nurse”.

Solution:

1. **Dot product:** $\vec{v}_{\text{doctor}} \cdot \vec{v}_{\text{nurse}} = 5(4) + 3(4) + 2(1) = 20 + 12 + 2 = 34$
2. **Magnitude of doctor:** $\|\vec{v}_{\text{doctor}}\| = \sqrt{5^2 + 3^2 + 2^2} = \sqrt{25 + 9 + 4} = \sqrt{38} \approx 6.16$
3. **Magnitude of nurse:** $\|\vec{v}_{\text{nurse}}\| = \sqrt{4^2 + 4^2 + 1^2} = \sqrt{16 + 16 + 1} = \sqrt{33} \approx 5.74$
4. **Cosine similarity:**

$$\text{cosine}(\text{doctor}, \text{nurse}) = \frac{34}{6.16 \times 5.74} = \frac{34}{35.40} \approx 0.96$$

Interpretation: Cosine similarity of 0.96 indicates these words are very similar in meaning (both are medical professionals).

Compare with unrelated word: $\vec{v}_{\text{pizza}} = (1, 0, 0)$

$$\text{cosine}(\text{doctor}, \text{pizza}) = \frac{5(1) + 3(0) + 2(0)}{6.16 \times 1} = \frac{5}{6.16} \approx 0.81$$

Even “pizza” has some similarity because of dimension overlap, but less than “nurse”.

2.6 Word Sense Induction (WSI)

Word Sense Induction (WSI) is the unsupervised counterpart to WSD. Instead of using a pre-defined inventory of senses (like WordNet), WSI algorithms automatically discover the senses of a word from corpus data. This is valuable because building sense-labeled corpora is expensive and time-consuming, and the optimal sense granularity may differ across applications.

Most WSI algorithms follow the early work of Schütze (1992, 1998) and use clustering over word embeddings:

1. **Training phase:** For each token of the target word in a corpus, compute a context vector representing the words that appear near the target word.
2. **Clustering:** Apply a clustering algorithm (such as agglomerative clustering or k-means) to group these context vectors into a predefined number of clusters. Each cluster represents an automatically induced sense.
3. **Sense representation:** Compute the centroid (average vector) of each cluster. This centroid becomes the sense vector representing that induced sense.

At test time, to disambiguate a new occurrence of the target word, we compute its context vector and assign it to the cluster whose centroid is closest, using cosine similarity or Euclidean distance as the distance metric.



Figure 5: Semantic Analysis framework visualization from source material (SemanticAnalysisPart2 PPT), showing the pipeline from raw text to semantic representations.

3 Module 3: Naive Bayes and Text Classification

3.1 Introduction to Text Classification

Text classification (also called text categorization) is the task of assigning a predefined label or category to an entire text or document based on its content. This is one of the most fundamental and widely applied tasks in natural language processing, forming the backbone of systems that must automatically organize, filter, or analyze large volumes of text.

The applications of text classification are extensive and touch nearly every domain where text is processed at scale:

- **Sentiment Analysis:** Determining whether a piece of text (such as a product review, tweet, or movie review) expresses a positive, negative, or neutral sentiment. This is crucial for businesses monitoring customer feedback, for political analysts tracking public opinion, and for financial traders gauging market sentiment.
- **Spam Detection:** Classifying incoming emails as spam or legitimate (ham). This is one of the oldest and most successful applications of text classification, saving billions of hours of human attention.
- **Language Identification:** Automatically determining which language a given piece of text is written in. This is essential for multilingual search engines, translation services, and content filtering systems.
- **Authorship Attribution:** Determining the author of a text based on stylistic features. This has applications in forensic linguistics, plagiarism detection, and literary analysis.
- **Subject Categorization:** Assigning library subject categories, news article topics, or patent classifications to documents. This enables efficient information retrieval and knowledge organization.

Text classification is typically formulated as a supervised learning problem: given a training set of documents labeled with their correct categories, we learn a function that maps new, unlabeled documents to categories.

3.2 Naive Bayes Classifiers

The **Naive Bayes classifier** is one of the simplest yet most effective text classification algorithms. Despite its simplifying assumptions, it often performs surprisingly well on text data and serves as a strong baseline against which more complex models are compared.

Naive Bayes is a **generative classifier**. Unlike discriminative classifiers (such as logistic regression or SVMs), which directly model the decision boundary between classes, generative classifiers build a model of how each class could have generated the observed data. The name “Naive Bayes” comes from two sources: “Bayes” because the classifier is derived from Bayes’ theorem, and “Naive” because it makes a strong simplifying assumption about the independence of features.

3.2.1 Bayes’ Rule and the Classification Decision

At the heart of the Naive Bayes classifier is **Bayes’ rule** (also called Bayes’ theorem), which relates the conditional and marginal probabilities of random events:

$$P(c \mid d) = \frac{P(d \mid c)P(c)}{P(d)}$$

In the context of text classification:

- c represents a class (e.g., “positive” or “negative” for sentiment analysis)
- d represents a document

- $P(c)$ is the **prior probability** of class c (how likely the class is before seeing the document)
- $P(d | c)$ is the **likelihood** (how likely the document is if the class is c)
- $P(c | d)$ is the **posterior probability** (how likely the class is after seeing the document)

Because $P(d)$ is the same for all classes, it does not affect which class is most probable. Therefore, the classifier simply returns the class with the maximum posterior:

$$\hat{c} = \arg \max_{c \in C} P(c | d) = \arg \max_{c \in C} P(d | c)P(c)$$

3.2.2 The Bag of Words Assumption

To make the computation of $P(d | c)$ tractable, the Naive Bayes classifier makes the **bag of words assumption**: a document is represented as an unordered set (or multiset) of words with their frequencies. The position of words in the document is completely ignored—only the identity and frequency of words matter.

For example, under the bag of words assumption, the sentences “The movie was good, not bad at all” and “The movie was bad, not good at all” would have identical representations, even though their meanings are opposite. This is clearly a severe simplification, but it works surprisingly well in practice for many classification tasks because the overall word frequencies often carry enough signal to distinguish classes.

3.2.3 The Naive Bayes Assumption: Conditional Independence

The second and more critical assumption is the **conditional independence assumption**: given the class label, the probability of each word occurring in the document is independent of the probabilities of all other words. Formally:

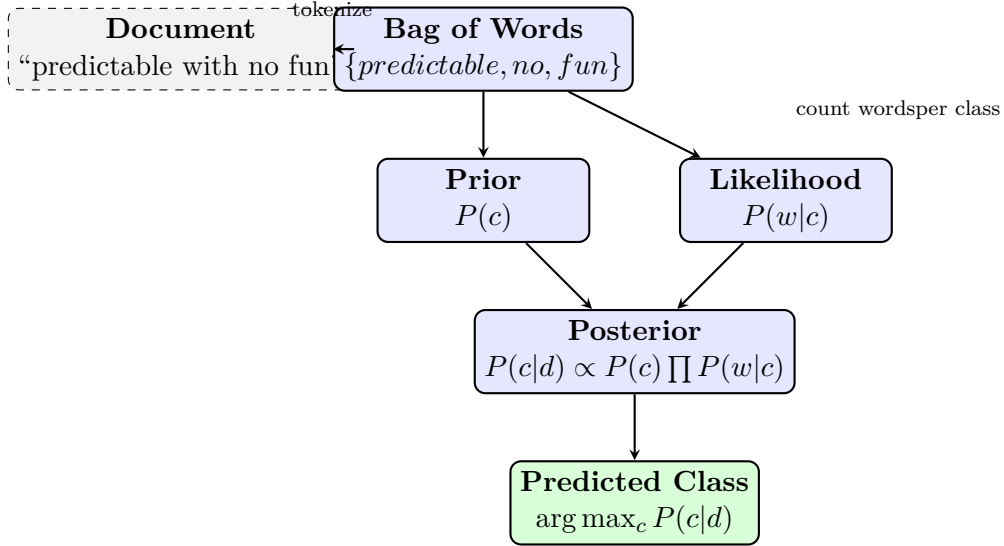
$$P(f_1, f_2, \dots, f_n | c) = P(f_1 | c) \times P(f_2 | c) \times \dots \times P(f_n | c)$$

This assumption is “naive” because in natural language, words are clearly not independent—“New” and “York” co-occur far more often than chance would predict, and “not” strongly influences the interpretation of nearby words. However, the conditional independence assumption dramatically reduces the number of parameters that must be estimated from data and often leads to robust classifiers that generalize well.

Combining the bag of words and conditional independence assumptions, the Naive Bayes classification rule becomes:

$$c_{NB} = \arg \max_{c \in C} P(c) \prod_{f \in F} P(f | c)$$

where F is the set of feature values (word occurrences) in the document, and f ranges over the words (or word features) in the document.



3.2.4 Training the Naive Bayes Classifier

Training a Naive Bayes classifier involves estimating the probabilities $P(c)$ and $P(f | c)$ from the training data using Maximum Likelihood Estimation with Laplace smoothing.

The **class prior** $P(c)$ is estimated as the proportion of training documents that belong to class c :

$$\hat{P}(c) = \frac{N_c}{N_{doc}}$$

where N_c is the number of training documents in class c and N_{doc} is the total number of training documents.

The **likelihood** $\hat{P}(w_i | c)$ is estimated from the word counts in the training documents of class c . To handle words that never appeared in the training documents of a particular class (which would cause the entire product to become zero), we use **add-one (Laplace) smoothing**:

$$\hat{P}(w_i | c) = \frac{\text{count}(w_i, c) + 1}{\sum_{w \in V} (\text{count}(w, c) + 1)} = \frac{\text{count}(w_i, c) + 1}{\sum_{w \in V} \text{count}(w, c) + |V|}$$

where $|V|$ is the vocabulary size. The $+1$ in the numerator ensures that every word has a non-zero probability, and the $+|V|$ in the denominator ensures that the probabilities sum to 1.

Numerical: Naive Bayes Sentiment Classification

Training Data:

Class	Document
-	just plain boring
-	entirely predictable and lacks energy
-	no surprises and very few laughs
+	very powerful
+	the most fun film of the summer

Test sentence: “predictable with no fun”

Step 1: Compute prior probabilities

$$P(-) = \frac{3}{5} = 0.6, \quad P(+) = \frac{2}{5} = 0.4$$

Step 2: Build vocabulary and compute word counts

Vocabulary (20 words): just, plain, boring, entirely, predictable, and, lacks, energy, no, surprises, very, few, laughs, powerful, the, most, fun, film, of, summer

Negative class total word count: 14 (just, plain, boring, entirely, predictable, and×2, lacks, energy, no, surprises, very, few, laughs)

Positive class total word count: 9 (very×2, powerful, the, most, fun, film, of, summer)

Step 3: Compute likelihoods with add-1 smoothing ($|V| = 20$)

For “predictable”:

$$P(\text{predictable} \mid -) = \frac{1+1}{14+20} = \frac{2}{34}, \quad P(\text{predictable} \mid +) = \frac{0+1}{9+20} = \frac{1}{29}$$

For “no”:

$$P(\text{no} \mid -) = \frac{1+1}{14+20} = \frac{2}{34}, \quad P(\text{no} \mid +) = \frac{0+1}{9+20} = \frac{1}{29}$$

For “fun”:

$$P(\text{fun} \mid -) = \frac{0+1}{14+20} = \frac{1}{34}, \quad P(\text{fun} \mid +) = \frac{1+1}{9+20} = \frac{2}{29}$$

Step 4: Compute posterior (ignoring “with” as unknown)

$$P(-) \cdot P(S \mid -) = \frac{3}{5} \times \frac{2}{34} \times \frac{2}{34} \times \frac{1}{34} = \frac{12}{5 \times 39304} \approx 6.1 \times 10^{-5}$$

$$P(+) \cdot P(S \mid +) = \frac{2}{5} \times \frac{1}{29} \times \frac{1}{29} \times \frac{2}{29} = \frac{4}{5 \times 24389} \approx 3.2 \times 10^{-5}$$

Result: Since $6.1 \times 10^{-5} > 3.2 \times 10^{-5}$, the negative class has higher posterior probability.

Prediction: **NEGATIVE** sentiment

Worked Example 19: Naive Bayes for Spam Detection

Training Data:

Class	Document
spam	buy cheap products now
spam	get rich quick click here
spam	limited offer buy now
ham	meeting scheduled for tomorrow
ham	please review the attached document
ham	lunch at noon today

Test: “buy now cheap products”

Step 1: Compute priors

$$P(\text{spam}) = \frac{3}{6} = 0.5, \quad P(\text{ham}) = \frac{3}{6} = 0.5$$

Step 2: Vocabulary and counts ($|V| = 18$)

Spam word counts: buy(2), cheap(1), products(1), now(2), get(1), rich(1), quick(1), click(1), here(1), limited(1), offer(1) = 13 total

Ham word counts: meeting(1), scheduled(1), for(1), tomorrow(1), please(1), review(1), the(1), attached(1), document(1), lunch(1), at(1), noon(1), today(1) = 13 total

Step 3: Compute likelihoods with add-1 smoothing

$$P(\text{buy}|\text{spam}) = \frac{2+1}{13+18} = \frac{3}{31}, \quad P(\text{buy}|\text{ham}) = \frac{0+1}{13+18} = \frac{1}{31}$$

$$P(\text{now}|\text{spam}) = \frac{2+1}{31} = \frac{3}{31}, \quad P(\text{now}|\text{ham}) = \frac{0+1}{31} = \frac{1}{31}$$

$$P(\text{cheap}|\text{spam}) = \frac{1+1}{31} = \frac{2}{31}, \quad P(\text{cheap}|\text{ham}) = \frac{0+1}{31} = \frac{1}{31}$$

$$P(\text{products}|\text{spam}) = \frac{1+1}{31} = \frac{2}{31}, \quad P(\text{products}|\text{ham}) = \frac{0+1}{31} = \frac{1}{31}$$

Step 4: Compute posteriors

$$P(\text{spam})P(S|\text{spam}) = 0.5 \times \frac{3}{31} \times \frac{3}{31} \times \frac{2}{31} \times \frac{2}{31} = 0.5 \times \frac{36}{923521} = 1.95 \times 10^{-5}$$

$$P(\text{ham})P(S|\text{ham}) = 0.5 \times \frac{1}{31} \times \frac{1}{31} \times \frac{1}{31} \times \frac{1}{31} = 0.5 \times \frac{1}{923521} = 5.41 \times 10^{-7}$$

Result: Spam class has much higher probability. Prediction: **SPAM**

Worked Example 20: Naive Bayes with Binary Features

Given: Binary multinomial Naive Bayes (clip counts at 1)

Training (same as Example 18, but binary):

-	just plain boring
-	entirely predictable and lacks energy
-	no surprises and very few laughs
+	very powerful
+	the most fun film of the summer

Test: “predictable with no fun”

Step 1: Binary feature representation — each word is present (1) or absent (0), regardless of frequency.

Step 2: Document frequencies (number of docs containing each word):

- Negative class (3 docs): predictable(1), no(1), fun(0), with(0)
- Positive class (2 docs): predictable(0), no(0), fun(1), with(0)

Step 3: Compute likelihoods with add-1 smoothing ($|V| = 20$)

$$P(\text{predictable}|-) = \frac{1+1}{3+20} = \frac{2}{23}, \quad P(\text{predictable}+) = \frac{0+1}{2+20} = \frac{1}{22}$$

$$P(\text{no}|-) = \frac{1+1}{23} = \frac{2}{23}, \quad P(\text{no}+) = \frac{0+1}{22} = \frac{1}{22}$$

$$P(\text{fun}|-) = \frac{0+1}{23} = \frac{1}{23}, \quad P(\text{fun}+) = \frac{1+1}{22} = \frac{2}{22}$$

Step 4: Posteriors (ignoring “with” as unknown)

$$P(-)P(S|-) = \frac{3}{5} \times \frac{2}{23} \times \frac{2}{23} \times \frac{1}{23} = \frac{12}{5 \times 12167} = 1.97 \times 10^{-4}$$

$$P(+)P(S|+) = \frac{2}{5} \times \frac{1}{22} \times \frac{1}{22} \times \frac{2}{22} = \frac{4}{5 \times 10648} = 7.51 \times 10^{-5}$$

Result: Negative class has higher probability. Prediction: **NEGATIVE**

Note: Binary NB gives more weight to rare words and handles documents with repeated words more robustly.

3.3 Optimizations for Sentiment Analysis

While the basic multinomial Naive Bayes classifier performs well, several optimizations have been developed specifically for sentiment analysis and other text classification tasks.

3.3.1 Binary Multinomial Naive Bayes

In standard multinomial Naive Bayes, word frequencies are counted—if a word appears 5 times in a document, it contributes 5 factors to the product. However, for sentiment analysis, **presence** often matters more than **frequency**. A review that mentions “bad” once and “terrible” once is more negative than one that mentions “bad” five times but no other negative words.

Binary multinomial Naive Bayes addresses this by clipping all word counts at 1. Each word is treated as either present or absent in the document, regardless of how many times it appears. This simple modification often improves classification accuracy for sentiment analysis because it reduces the influence of repeated words and focuses the model on the vocabulary of the document.

3.3.2 Negation Handling

One of the most important challenges in sentiment analysis is handling **negation**. Words like “not” and “no” can completely flip the sentiment of nearby words. For example, “not good” should be classified as negative, even though “good” is a positive word.

A simple but effective heuristic for handling negation is to prepend a “NOT” *prefix to every word that appears between an*

3.3.3 Sentiment Lexicons

Sentiment lexicons are pre-compiled dictionaries of words annotated with their sentiment polarity (positive, negative, or neutral). Incorporating lexicon features into a Naive Bayes classifier can improve performance, especially when training data is limited. Well-known sentiment lexicons include:

- **General Inquirer:** A general-purpose lexicon with positive/negative annotations for thousands of English words.
- **LIWC (Linguistic Inquiry and Word Count):** A commercial lexicon categorizing words into psychological and linguistic categories.
- **MPQA Subjectivity Lexicon:** A freely available lexicon of subjective words annotated with polarity.

3 Step 1: Compute Prior Probabilities

$$P(\text{Spam}) = \frac{2}{4} = 0.5$$
$$P(\text{Ham}) = \frac{2}{4} = 0.5$$

4 Step 2: Build Vocabulary

All unique words:

V

$= \{b, w, y, c, h, e, a, p, m, e, d, s, o, n, l, i, n, e, m, e, e, t, i, n, g, s, s, c, h, e, d, u, l, e, t, o, d, a, y, p, r, o, j, e, c, t\}$

Total vocabulary size = 8

Figure 6: Naive Bayes Classifier architecture and text classification pipeline from source material (Text Classification using Naive Bayes PPT), illustrating the bag-of-words representation and probability estimation process.

4 Module 4: Deep Learning Architectures for Sequence Processing

4.1 Recurrent Neural Networks (RNNs)

Recurrent Neural Networks (RNNs) are a class of artificial neural networks specifically designed to process sequential data. Unlike feedforward neural networks, which assume that all inputs are independent of each other, RNNs maintain an internal memory state that captures information about previous inputs in the sequence. This makes them uniquely suited for tasks involving natural language, speech, time series, and any other domain where the order of data matters.

The fundamental challenge that RNNs address is that standard feedforward networks require fixed-size inputs. A sentence of length 5 and a sentence of length 50 cannot both be fed into the same fixed-input network without some form of truncation, padding, or windowing. RNNs solve this by processing the sequence one element at a time, maintaining a hidden state that evolves as the network reads each new element. This allows RNNs to handle variable-length sequences naturally and to capture dependencies between elements at different positions in the sequence.

4.1.1 RNN Architecture

The architecture of a simple RNN consists of three sets of learnable weight matrices and a hidden state vector:

- W_x : The **input-to-hidden** weight matrix, which transforms the input at the current time step into the hidden state space.
- W_h : The **hidden-to-hidden** (recurrent) weight matrix, which transforms the previous hidden state into the new hidden state. This is the key matrix that enables the network to maintain memory across time steps.
- W_o : The **hidden-to-output** weight matrix, which transforms the hidden state into the output space (e.g., a probability distribution over vocabulary words for language modeling).

At each time step t , the RNN receives an input vector x_t (typically a word embedding) and updates its hidden state h_t according to the following recurrence relation:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b_h)$$

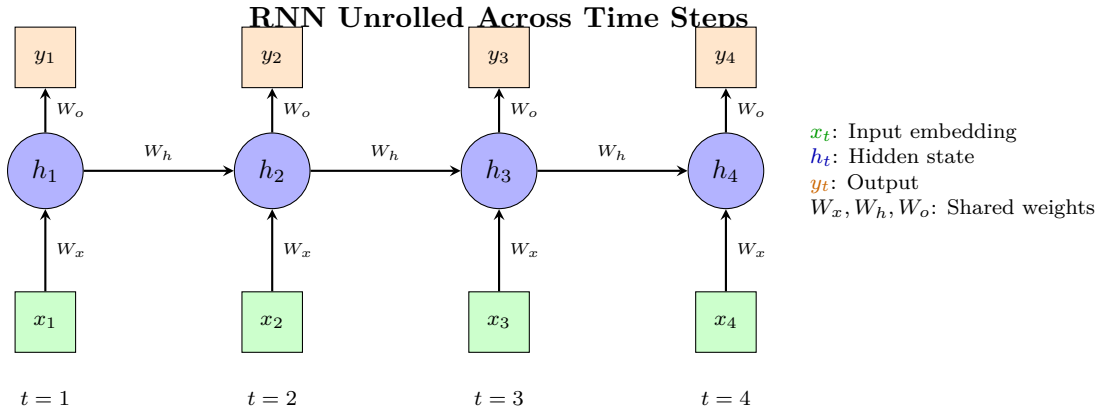
where b_h is a bias vector and $\tanh$ is the hyperbolic tangent activation function, which squashes the values to the range $[-1, 1]$. The hidden state h_t thus serves as a compressed representation of all information the network has seen in the sequence up to time t .

The output at each time step y_t is computed from the hidden state:

$$y_t = \text{softmax}(W_o h_t + b_o)$$

where softmax converts the output into a probability distribution over the vocabulary, and b_o is the output bias vector.

The three weight matrices (W_x , W_h , W_o) are shared across all time steps—the same parameters are used whether processing the first word or the hundredth word. This **parameter sharing** is what makes RNNs scalable to long sequences: the number of parameters is fixed regardless of sequence length.



RNN Memory Mechanism: Numerical Example

Given: A simple character-level RNN with $h_t = \tanh(W_h h_{t-1} + W_x x_t)$. Let $W_h = 0.5$, $W_x = 1.0$, initial $h_0 = 0$.

Input sequence: $x = [1, 0, 1, 0]$ (binary features).

Compute hidden states:

1. $h_1 = \tanh(0.5(0) + 1.0(1)) = \tanh(1.0) = 0.762$
2. $h_2 = \tanh(0.5(0.762) + 1.0(0)) = \tanh(0.381) = 0.363$
3. $h_3 = \tanh(0.5(0.363) + 1.0(1)) = \tanh(1.182) = 0.829$
4. $h_4 = \tanh(0.5(0.829) + 1.0(0)) = \tanh(0.414) = 0.392$

Interpretation: Even though $x_2 = x_4 = 0$, the hidden states h_2 and h_4 are non-zero because they carry memory from previous time steps. The hidden state acts as a compressed representation of the sequence history.

4.1.2 Training RNNs: Backpropagation Through Time (BPTT)

RNNs are trained using a variant of backpropagation called **Backpropagation Through Time (BPTT)**. The key idea is to “unroll” the RNN across time steps, creating a computational graph where each time step is a layer in a very deep feedforward network. The loss is computed at each time step (for sequence labeling) or at the final time step (for sequence classification), and gradients are propagated backward through this unrolled graph.

The training procedure works as follows:

1. **Forward pass:** For each time step $t = 1, 2, \dots, T$, compute h_t and y_t using the recurrence relation. Accumulate the loss at each step.
2. **Backward pass:** Starting from the final time step T and moving backward to $t = 1$, compute the gradients of the loss with respect to each parameter. Because h_t depends on h_{t-1} , the gradient at time t must flow backward through all previous time steps.
3. **Parameter update:** After computing gradients for all time steps, update the shared parameters W_x , W_h , and W_o using gradient descent (or a variant such as Adam).

4.1.3 RNN as Language Model

When used as a **language model**, an RNN predicts the next word in a sequence given the previous words. At each time step t , the input x_t is the embedding of the current word, and the target output is the embedding (or one-hot vector) of the next word. The network is trained to minimize

the cross-entropy loss between its predicted probability distribution and the actual next word:

$$L_t = -\log P(w_{t+1} \mid w_1, w_2, \dots, w_t)$$

The total loss for a sequence is the sum (or average) of the losses at all time steps:

$$L = \sum_{t=1}^T L_t = -\sum_{t=1}^T \log P(w_{t+1} \mid w_1, \dots, w_t)$$

4.1.4 Autoregressive Generation

Once trained, an RNN language model can **generate** new text by sampling from its output distribution. The process, called **autoregressive generation**, works as follows:

1. Start with a special start-of-sequence token $\langle s \rangle$.
2. Feed the start token into the RNN and sample the first output word from the softmax distribution.
3. Feed the sampled word back into the RNN as the next input.
4. Repeat until the end-of-sequence token $\langle /s \rangle$ is generated or a maximum length is reached.

This process can produce novel sentences that mimic the style and content of the training data, though the quality depends heavily on the model architecture, training data, and decoding strategy (greedy, beam search, or sampling).

4.2 Sequence Labeling and Sequence Classification

RNNs can be applied to two broad categories of sequence tasks:

Sequence labeling (also called sequence tagging) assigns a label to each element of the sequence. The output at each time step t is a label for the input at that time step. Examples include:

- **Part-of-Speech (POS) tagging:** Label each word with its grammatical category (noun, verb, adjective, etc.).
- **Named Entity Recognition (NER):** Label each word with its entity type (person, organization, location, etc.).

For sequence labeling, the loss is computed and backpropagated at every time step.

Sequence classification assigns a single label to the entire sequence. The input is a sequence of words, and the output is a single class label. Examples include:

- **Sentiment analysis:** Classify a movie review as positive or negative.
- **Spam detection:** Classify an email as spam or legitimate.

For sequence classification, we typically use the final hidden state h_T (or a pooling of all hidden states) as the representation of the entire sequence, and feed it through a classification layer to produce the output label.

4.3 Vanishing and Exploding Gradients

A major challenge in training RNNs is the **vanishing and exploding gradients problem**. During BPTT, the gradient at time step t must flow backward through the recurrent connections to all previous time steps. This involves repeatedly multiplying by the weight matrix W_h and by the derivative of the activation function.

When the eigenvalues of W_h are small (or when using activation functions like tanh whose derivatives are less than 1), the gradient becomes exponentially smaller as it propagates backward

through time. This means that the network cannot effectively learn long-range dependencies—the influence of words at the beginning of a sentence on words at the end is negligible.

Conversely, when the eigenvalues of W_h are large, the gradient can grow exponentially, causing **exploding gradients** that make training unstable.

Several solutions have been proposed:

- **Gradient clipping:** When the gradient norm exceeds a threshold, rescale it to prevent explosion.
- **Better initialization:** Initialize W_h with small values or orthogonal matrices to keep eigenvalues near 1.
- **Use ReLU:** The ReLU activation has derivative 1 (for positive inputs), which mitigates vanishing gradients compared to tanh or sigmoid.
- **Architectural solutions:** Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) architectures were specifically designed to solve the vanishing gradient problem through gating mechanisms.

4.4 Long Short-Term Memory (LSTM)

Long Short-Term Memory (LSTM) networks are a special kind of RNN designed to overcome the vanishing gradient problem and to learn long-term dependencies more effectively. Introduced by Hochreiter and Schmidhuber in 1997, LSTMs have become the default choice for many sequence modeling tasks.

The key innovation of the LSTM is the **cell state** (also called the constant error carousel), which acts as a transport highway that runs straight through the entire sequence with only minor linear interactions. Information can flow along the cell state virtually unchanged across many time steps, which allows the network to preserve information over long distances.

To regulate the flow of information into and out of the cell state, the LSTM uses three **gates**, each consisting of a sigmoid neural network layer and a pointwise multiplication operation:

4.4.1 The Forget Gate

The **forget gate** decides what information to discard from the cell state. It looks at the previous hidden state h_{t-1} and the current input x_t , and outputs a number between 0 and 1 for each element of the cell state. A value of 0 means “completely forget this information,” while 1 means “completely keep this information.”

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

4.4.2 The Input Gate

The **input gate** decides what new information to store in the cell state. It has two parts: a sigmoid layer that decides which values to update, and a tanh layer that creates candidate values that could be added to the cell state.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

The cell state is then updated by combining the forget gate and input gate:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

where $\odot$ denotes element-wise multiplication.

4.4.3 The Output Gate

The **output gate** decides what the next hidden state h_t should be. The hidden state is a filtered version of the cell state, where the filter is determined by a sigmoid layer:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \odot \tanh(C_t)$$

The gating mechanism is what makes LSTMs so effective: by learning which information to forget, which to store, and which to output, the network can maintain a stable gradient flow across hundreds or even thousands of time steps.

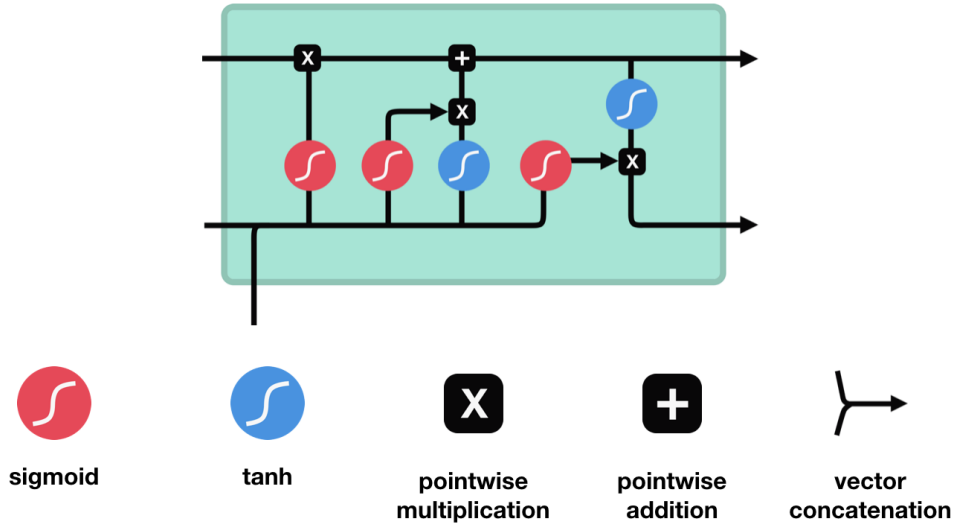


Figure 7: RNN Architecture and LSTM cell structure from source material (Chapter 4 PPT), showing the unrolled recurrent network and the internal gating mechanisms of the LSTM cell.

4.5 Gated Recurrent Unit (GRU)

The **Gated Recurrent Unit (GRU)**, introduced by Cho et al. in 2014, is a simplified variant of the LSTM that achieves similar performance with fewer parameters and faster training. The GRU merges the forget and input gates into a single **update gate** and eliminates the separate cell state, using only the hidden state to carry information across time steps.

The GRU has two gates:

4.5.1 The Reset Gate

The **reset gate** r_t determines how much of the previous hidden state to forget when computing the candidate hidden state. If r_t is close to 0, the past hidden state is largely ignored, and the candidate state depends mainly on the current input. If r_t is close to 1, the past hidden state is fully preserved.

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

4.5.2 The Update Gate

The **update gate** z_t determines how much of the previous hidden state to keep and how much of the new candidate hidden state to use. It is analogous to the combination of the forget and input gates in an LSTM.

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

The candidate hidden state is computed as:

$$\tilde{h}_t = \tanh(W \cdot [r_t \odot h_{t-1}, x_t])$$

And the final hidden state is a weighted combination of the old and candidate states:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

Key Insight: Interpreting the GRU Update Gate

The update gate z_t controls the balance between old and new information:

- **If $z_t \approx 0$:** Then $1 - z_t \approx 1$, so most of the old hidden state h_{t-1} is preserved, and only a small amount of new information from $\tilde{h}_t$ is incorporated. This is useful when the current input is not very informative or when we want to maintain long-term memory.
- **If $z_t \approx 1$:** Then $1 - z_t \approx 0$, so the old hidden state is largely discarded, and the new candidate state $\tilde{h}_t$ dominates. This allows the network to rapidly update its memory when important new information arrives.

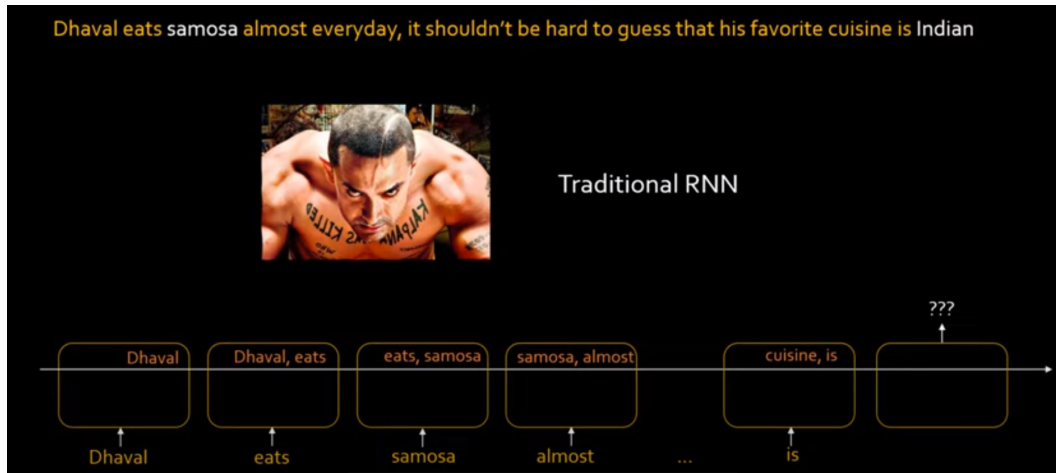


Figure 8: GRU and LSTM cell diagrams from source material (Gated Recurrent Unit PPT), comparing the internal structure and gating mechanisms of GRU and LSTM architectures.

Worked Example 21: RNN Forward Pass and Cross-Entropy Loss

Given: Sentence “The cat sat”, vocabulary = {The, cat, sat, on, mat, dog}
 Network parameters (simplified for 2D hidden state, vocabulary size 6):

$$W_x = \begin{bmatrix} 0.1 & 0.2 & 0.3 & 0.1 & 0.2 & 0.0 \\ 0.0 & 0.1 & 0.2 & 0.3 & 0.1 & 0.2 \end{bmatrix}, \quad W_h = \begin{bmatrix} 0.5 & -0.1 \\ 0.1 & 0.4 \end{bmatrix}$$

$$W_o = \begin{bmatrix} 0.2 & 0.1 \\ 0.1 & 0.3 \\ 0.3 & 0.0 \\ 0.0 & 0.2 \\ 0.1 & 0.1 \\ 0.2 & 0.2 \end{bmatrix}, \quad b_h = \begin{bmatrix} 0.0 \\ 0.0 \end{bmatrix}, \quad b_o = \begin{bmatrix} 0.0 \\ 0.0 \\ 0.0 \\ 0.0 \\ 0.0 \\ 0.0 \end{bmatrix}$$

Input “The” = one-hot index 0: $x_1 = [1, 0, 0, 0, 0, 0]^T$

Step 1: Compute hidden state h_1

$$h_1 = \tanh(W_h h_0 + W_x x_1 + b_h) = \tanh(W_x x_1) = \tanh\left(\begin{bmatrix} 0.1 \\ 0.0 \end{bmatrix}\right) = \begin{bmatrix} 0.0997 \\ 0.0 \end{bmatrix}$$

Step 2: Compute output logits

$$o_1 = W_o h_1 + b_o = \begin{bmatrix} 0.2 \times 0.0997 \\ 0.1 \times 0.0997 \\ 0.3 \times 0.0997 \\ 0.0 \times 0.0997 \\ 0.1 \times 0.0997 \\ 0.2 \times 0.0997 \end{bmatrix} = \begin{bmatrix} 0.0199 \\ 0.0100 \\ 0.0299 \\ 0.0 \\ 0.0100 \\ 0.0199 \end{bmatrix}$$

Step 3: Apply softmax

$$\text{sum} = e^{0.0199} + e^{0.0100} + e^{0.0299} + e^0 + e^{0.0100} + e^{0.0199} = 1.0201 + 1.0101 + 1.0304 + 1.0 + 1.0101 + 1.0201 = 6.0908$$

$$P(\text{The}) = \frac{1.0201}{6.0908} = 0.1675, \quad P(\text{cat}) = \frac{1.0101}{6.0908} = 0.1658$$

$$P(\text{sat}) = \frac{1.0304}{6.0908} = 0.1692 \text{ (highest)}$$

Step 4: Cross-entropy loss for target “cat” (index 1)

$$L_1 = -\log P(\text{cat}) = -\log(0.1658) = 1.797$$

Step 5: Repeat for $t = 2$ (input “cat”, target “sat”) and $t = 3$ (input “sat”, target “</s>”).

Total loss = $L_1 + L_2 + L_3$

Worked Example 22: LSTM Gate Computations

Given: Input $x_t = [0.5, -0.3]^T$, previous hidden state $h_{t-1} = [0.2, 0.1]^T$, previous cell state $C_{t-1} = [0.8, -0.2]^T$

Weight matrices (simplified, concatenated input and hidden):

$$W_f = \begin{bmatrix} 0.1 & -0.2 & 0.3 & 0.1 \\ 0.2 & 0.1 & -0.1 & 0.3 \end{bmatrix}, \quad W_i = \begin{bmatrix} 0.2 & 0.1 & 0.1 & -0.2 \\ -0.1 & 0.3 & 0.2 & 0.1 \end{bmatrix}$$

$$W_C = \begin{bmatrix} 0.1 & 0.2 & -0.1 & 0.3 \\ 0.3 & -0.1 & 0.2 & 0.1 \end{bmatrix}, \quad W_o = \begin{bmatrix} 0.2 & -0.1 & 0.1 & 0.3 \\ 0.1 & 0.3 & -0.2 & 0.1 \end{bmatrix}$$

All biases = $[0, 0]^T$

Step 1: Concatenate input and hidden state

$$[h_{t-1}; x_t] = [0.2, 0.1, 0.5, -0.3]^T$$

Step 2: Compute forget gate

$$f_t = \sigma(W_f[h_{t-1}; x_t]) = \sigma \left(\begin{bmatrix} 0.1(0.2) - 0.2(0.1) + 0.3(0.5) + 0.1(-0.3) \\ 0.2(0.2) + 0.1(0.1) - 0.1(0.5) + 0.3(-0.3) \end{bmatrix} \right)$$

$$f_t = \sigma \left(\begin{bmatrix} 0.02 - 0.02 + 0.15 - 0.03 \\ 0.04 + 0.01 - 0.05 - 0.09 \end{bmatrix} \right) = \sigma \left(\begin{bmatrix} 0.12 \\ -0.09 \end{bmatrix} \right) = \begin{bmatrix} 0.530 \\ 0.478 \end{bmatrix}$$

Step 3: Compute input gate and candidate cell state

$$i_t = \sigma(W_i[h_{t-1}; x_t]) = \sigma \left(\begin{bmatrix} 0.2(0.2) + 0.1(0.1) + 0.1(0.5) - 0.2(-0.3) \\ -0.1(0.2) + 0.3(0.1) + 0.2(0.5) + 0.1(-0.3) \end{bmatrix} \right)$$

$$i_t = \sigma \left(\begin{bmatrix} 0.04 + 0.01 + 0.05 + 0.06 \\ -0.02 + 0.03 + 0.10 - 0.03 \end{bmatrix} \right) = \sigma \left(\begin{bmatrix} 0.16 \\ 0.08 \end{bmatrix} \right) = \begin{bmatrix} 0.540 \\ 0.520 \end{bmatrix}$$

$$\tilde{C}_t = \tanh(W_C[h_{t-1}; x_t]) = \tanh \left(\begin{bmatrix} 0.1(0.2) + 0.2(0.1) - 0.1(0.5) + 0.3(-0.3) \\ 0.3(0.2) - 0.1(0.1) + 0.2(0.5) + 0.1(-0.3) \end{bmatrix} \right)$$

$$\tilde{C}_t = \tanh \left(\begin{bmatrix} 0.02 + 0.02 - 0.05 - 0.09 \\ 0.06 - 0.01 + 0.10 - 0.03 \end{bmatrix} \right) = \tanh \left(\begin{bmatrix} -0.10 \\ 0.12 \end{bmatrix} \right) = \begin{bmatrix} -0.100 \\ 0.119 \end{bmatrix}$$

Step 4: Update cell state

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

$$C_t = \begin{bmatrix} 0.530 \times 0.8 \\ 0.478 \times (-0.2) \end{bmatrix} + \begin{bmatrix} 0.540 \times (-0.100) \\ 0.520 \times 0.119 \end{bmatrix}$$

$$C_t = \begin{bmatrix} 0.424 \\ -0.096 \end{bmatrix} + \begin{bmatrix} -0.054 \\ 0.062 \end{bmatrix} = \begin{bmatrix} 0.370 \\ -0.034 \end{bmatrix}$$

Step 5: Compute output gate and hidden state

$$o_t = \sigma(W_o[h_{t-1}; x_t]) = \sigma \left(\begin{bmatrix} 0.2(0.2) - 0.1(0.1) + 0.1(0.5) + 0.3(-0.3) \\ 0.1(0.2) + 0.3(0.1) - 0.2(0.5) + 0.1(-0.3) \end{bmatrix} \right)$$

$$o_t = \sigma \left(\begin{bmatrix} 0.04 - 0.01 + 0.05 - 0.09 \\ 0.02 + 0.03 - 0.10 - 0.03 \end{bmatrix} \right) = \sigma \left(\begin{bmatrix} -0.01 \\ -0.08 \end{bmatrix} \right) = \begin{bmatrix} 0.498 \\ 0.480 \end{bmatrix}$$

$$h_t = o_t \odot \tanh(C_t) = \begin{bmatrix} 0.498 \\ 0.480 \end{bmatrix} \odot \tanh \left(\begin{bmatrix} 0.370 \\ -0.034 \end{bmatrix} \right) = \begin{bmatrix} 0.498 \times 0.354 \\ 0.480 \times (-0.034) \end{bmatrix}$$

$$h_t = \begin{bmatrix} 0.176 \\ -0.016 \end{bmatrix}$$

Worked Example 23: GRU Gate Computations

Given: Input $x_t = [0.4, -0.2]^T$, previous hidden state $h_{t-1} = [0.3, 0.1]^T$

Weight matrices:

$$W_r = \begin{bmatrix} 0.2 & -0.1 & 0.3 & 0.1 \\ -0.1 & 0.2 & 0.1 & 0.3 \end{bmatrix}, \quad W_z = \begin{bmatrix} 0.1 & 0.2 & -0.1 & 0.3 \\ 0.3 & -0.1 & 0.2 & 0.1 \end{bmatrix}, \quad W = \begin{bmatrix} 0.2 & 0.1 & 0.1 & -0.2 \\ -0.1 & 0.3 & 0.2 & 0.1 \end{bmatrix}$$

All biases = $[0, 0]^T$

Step 1: Concatenate

$$[h_{t-1}; x_t] = [0.3, 0.1, 0.4, -0.2]^T$$

Step 2: Compute reset gate

$$r_t = \sigma(W_r[h_{t-1}; x_t]) = \sigma \left(\begin{bmatrix} 0.2(0.3) - 0.1(0.1) + 0.3(0.4) + 0.1(-0.2) \\ -0.1(0.3) + 0.2(0.1) + 0.1(0.4) + 0.3(-0.2) \end{bmatrix} \right)$$

$$r_t = \sigma \left(\begin{bmatrix} 0.06 - 0.01 + 0.12 - 0.02 \\ -0.03 + 0.02 + 0.04 - 0.06 \end{bmatrix} \right) = \sigma \left(\begin{bmatrix} 0.15 \\ -0.03 \end{bmatrix} \right) = \begin{bmatrix} 0.537 \\ 0.493 \end{bmatrix}$$

Step 3: Compute update gate

$$z_t = \sigma(W_z[h_{t-1}; x_t]) = \sigma \left(\begin{bmatrix} 0.1(0.3) + 0.2(0.1) - 0.1(0.4) + 0.3(-0.2) \\ 0.3(0.3) - 0.1(0.1) + 0.2(0.4) + 0.1(-0.2) \end{bmatrix} \right)$$

$$z_t = \sigma \left(\begin{bmatrix} 0.03 + 0.02 - 0.04 - 0.06 \\ 0.09 - 0.01 + 0.08 - 0.02 \end{bmatrix} \right) = \sigma \left(\begin{bmatrix} -0.05 \\ 0.14 \end{bmatrix} \right) = \begin{bmatrix} 0.488 \\ 0.535 \end{bmatrix}$$

Step 4: Compute candidate hidden state

$$r_t \odot h_{t-1} = \begin{bmatrix} 0.537 \times 0.3 \\ 0.493 \times 0.1 \end{bmatrix} = \begin{bmatrix} 0.161 \\ 0.049 \end{bmatrix}$$

Concatenate $[r_t \odot h_{t-1}; x_t] = [0.161, 0.049, 0.4, -0.2]^T$

$$\tilde{h}_t = \tanh(W[r_t \odot h_{t-1}; x_t]) = \tanh \left(\begin{bmatrix} 0.2(0.161) + 0.1(0.049) + 0.1(0.4) - 0.2(-0.2) \\ -0.1(0.161) + 0.3(0.049) + 0.2(0.4) + 0.1(-0.2) \end{bmatrix} \right)$$

$$\tilde{h}_t = \tanh \left(\begin{bmatrix} 0.0322 + 0.0049 + 0.04 + 0.04 \\ -0.0161 + 0.0147 + 0.08 - 0.02 \end{bmatrix} \right) = \tanh \left(\begin{bmatrix} 0.117 \\ 0.0586 \end{bmatrix} \right) = \begin{bmatrix} 0.117 \\ 0.0585 \end{bmatrix}$$

Step 5: Compute new hidden state

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

$$h_t = \begin{bmatrix} (1 - 0.488) \times 0.3 \\ (1 - 0.535) \times 0.1 \end{bmatrix} + \begin{bmatrix} 0.488 \times 0.117 \\ 0.535 \times 0.0585 \end{bmatrix}$$

$$h_t = \begin{bmatrix} 0.154 \\ 0.047 \end{bmatrix} + \begin{bmatrix} 0.057 \\ 0.031 \end{bmatrix} = \begin{bmatrix} 0.211 \\ 0.078 \end{bmatrix}$$

Interpretation: The update gate $z_t = [0.488, 0.535]$ means roughly 49-54% of the new candidate state is used, while 51-46% of the old state is retained.

4.6 Bidirectional Recurrent Neural Networks (Bi-RNN)

A standard RNN processes a sequence in one direction only—from left to right (or from past to future). This means that the prediction at time t can only use information from positions $1, 2, \dots, t$. For many tasks, however, it would be useful to also use information from the future context.

A **Bidirectional RNN (Bi-RNN)** addresses this limitation by processing the input sequence in **both directions simultaneously**. It consists of two separate hidden layers:

- A **forward layer** that processes the sequence from left to right, computing hidden states $\vec{h}_1, \vec{h}_2, \dots, \vec{h}_T$.
- A **backward layer** that processes the sequence from right to left, computing hidden states $\overleftarrow{h}_T, \overleftarrow{h}_{T-1}, \dots, \overleftarrow{h}_1$.

The final representation at each time step is typically the concatenation of the forward and backward hidden states: $h_t = [\vec{h}_t; \overleftarrow{h}_t]$.

Bi-RNNs are particularly effective for tasks where the full context matters, such as:

- **Sentiment analysis:** Words at the end of a sentence may change the sentiment of words at the beginning (e.g., “The movie started slowly but ended brilliantly”).
- **Named entity recognition:** Determining whether “Washington” is a person or a location requires seeing the surrounding context on both sides.
- **Machine translation:** The meaning of a word in the source language may depend on words both before and after it.

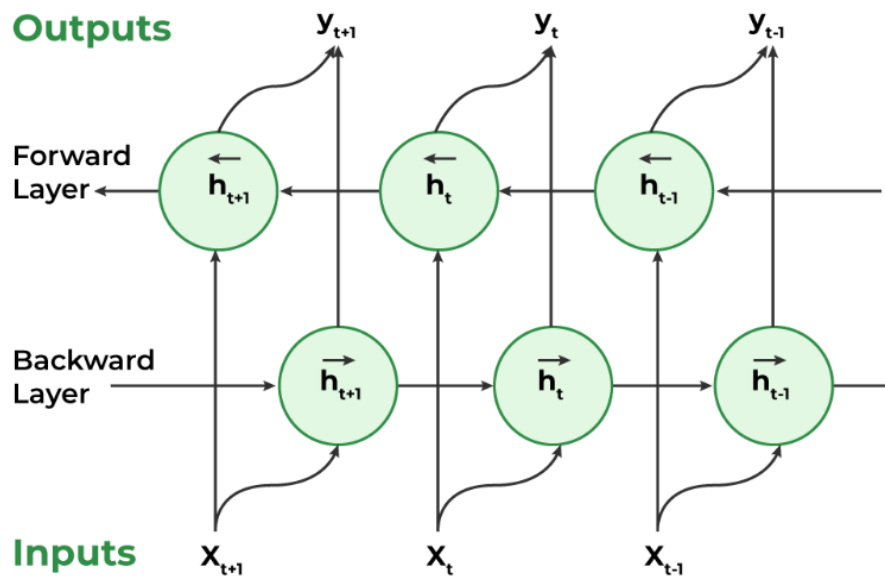
4.6.1 Advantages and Disadvantages of Bi-RNNs

Advantages:

- Access to both past and future context leads to more accurate predictions.
- Better handling of long-range dependencies because each layer only has to model half the sequence.
- The two layers provide a form of implicit regularization, reducing overfitting.

Disadvantages:

- **Higher computational cost:** Two separate recurrent passes are required, roughly doubling computation time.
- **Longer training time:** More parameters and more computation mean slower training.
- **Difficulty in parallelization:** The sequential nature of both forward and backward passes makes parallelization challenging.
- **Cannot be used for real-time generation:** Because the backward pass requires seeing the entire sequence, Bi-RNNs cannot be used for tasks like real-time speech recognition or live text generation.



Bi-directional Recurrent Neural Network

Figure 9: Bidirectional RNN architecture from source material (Bidirectional RNN PPT), illustrating how forward and backward layers process the sequence in opposite directions and combine their outputs.

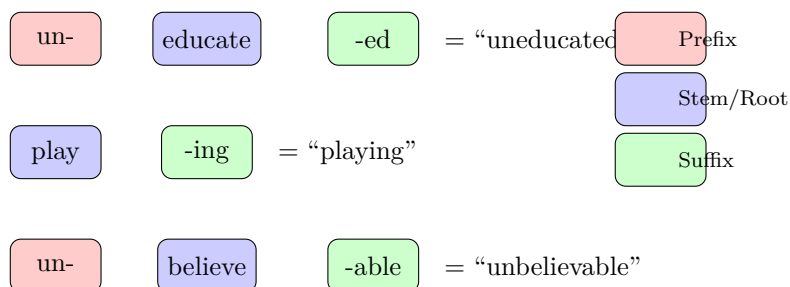
5 Module 5: Morphology and Syntax

5.1 Morphology

Morphology is the branch of linguistics that studies the internal structure of words and how words are formed from smaller meaningful units called morphemes. Understanding morphology is essential for many NLP tasks, including stemming, lemmatization, part-of-speech tagging, machine translation, and information retrieval, because it reveals the underlying structure and meaning of words that surface forms alone do not capture.

For example, the word “uneducated” can be broken down into three morphemes: “un-” (a prefix meaning “not”), “educate” (the root or stem), and “-d” (a suffix indicating past participle). Similarly, the word “playing” consists of “play” (the stem) and “-ing” (a suffix indicating present participle). By decomposing words into their morphemic components, we can understand relationships between words, recognize inflected forms, and build more efficient vocabulary representations.

Morphological Analysis Examples



Worked Example: Morpheme Identification

Task: Identify and classify all morphemes in the following words.

1. “**disagreement**”

- Prefix: “dis-” (negation)
- Stem: “agree” (core meaning)
- Suffix: “-ment” (nominalization - turns verb into noun)
- Total: 3 morphemes

2. “**unhappiness**”

- Prefix: “un-” (negation)
- Stem: “happy” (core meaning)
- Suffix: “-ness” (nominalization - turns adjective into noun)
- Total: 3 morphemes

3. “**books**”

- Stem: “book” (core meaning)
- Inflectional suffix: “-s” (plural marker)
- Total: 2 morphemes

Key Distinction: “-ment” and “-ness” are **derivational** morphemes (change word class), while “-s” is **inflectional** (only changes grammatical form).

5.1.1 Morphemes

A **morpheme** is the smallest unit of language that carries meaning. Morphemes cannot be broken down into smaller meaningful parts. For example, the word “cats” contains two morphemes: “cat” (meaning a feline animal) and “-s” (meaning plural). Neither “ca” nor “ts” carries independent meaning.

The study of morphemes is fundamental because it explains how a finite set of morphemes can generate an infinite variety of words through systematic combination rules.

5.1.2 Types of Morphemes

Morphemes are broadly classified into two categories based on whether they can stand alone as words:

1. **Free Morphemes:** These can appear as independent words and carry meaning on their own. Examples include “house”, “walk”, “of”, and “the”. When a free morpheme combines with bound morphemes, it is called the **stem** or **root** of the word.
2. **Bound Morphemes:** These cannot stand alone as words; they must attach to free morphemes. Examples include “-s” (as in “dogs”), “-ly” (as in “quickly”), and “un-” (as in “undo”). Bound morphemes are further divided into:
 - **Prefixes:** Attach before the stem (e.g., “un-”, “re-”, “pre-”)
 - **Suffixes:** Attach after the stem (e.g., “-ing”, “-ed”, “-s”)

- **Infixes:** Insert inside the stem (rare in English, common in some other languages)

5.1.3 Lexical vs. Grammatical Morphemes

Another important distinction is between lexical and grammatical morphemes:

Lexical morphemes (also called **content morphemes**) carry the core semantic content of a sentence. They belong to the “open classes” of a language—categories to which new words are regularly added. The major lexical categories are:

- **Nouns:** Words referring to entities (e.g., “dog”, “happiness”, “Google”)
- **Verbs:** Words referring to actions or states (e.g., “run”, “think”, “Skype”)
- **Adjectives:** Words describing properties (e.g., “red”, “tall”)
- **Adverbs:** Words modifying verbs, adjectives, or other adverbs (e.g., “quickly”, “very”)

Because these are open classes, new words are constantly added to the language. Words like “Google”, “blog”, “selfie”, and “hashtag” have all entered English in recent decades as new lexical morphemes.

Grammatical morphemes (also called **function morphemes**) belong to the “closed classes”—categories with a fixed, limited membership that rarely admits new words. They serve to tie the content words together into coherent grammatical structures. Examples include:

- **Conjunctions:** “and”, “but”, “or”, “because”
- **Prepositions:** “in”, “on”, “at”, “by”, “with”
- **Articles:** “the”, “a”, “an”
- **Pronouns:** “I”, “you”, “he”, “she”, “it”, “they”

5.1.4 Inflectional vs. Derivational Morphology

Morphemes can also be classified by the type of change they produce:

Inflectional morphology changes the grammatical form of a word without changing its word class (part of speech). English has only eight inflectional morphemes, all of which are suffixes:

- **Plural:** “-s” / “-es” (cat → cats)
- **Possessive:** “-’s” (John → John’s)
- **Third person singular present:** “-s” (walk → walks)
- **Past tense:** “-ed” (walk → walked)
- **Progressive:** “-ing” (walk → walking)
- **Past participle:** “-ed” / “-en” (eat → eaten)
- **Comparative:** “-er” (tall → taller)
- **Superlative:** “-est” (tall → tallest)

Inflections can be:

- **Regular:** Follow standard, predictable patterns (walked, played, cats)
- **Irregular:** Do not follow standard patterns (go → went, child → children, wife → wives)
- **Suppletion:** Use a completely different morpheme (good → better, bad → worse, go → went)

Derivational morphology creates new words, often changing the word class:

- **Class-changing:** care (verb) → careful (adjective) → carelessly (adverb)
- **Class-maintaining:** child (noun) → childhood (noun), friend (noun) → friendship (noun)

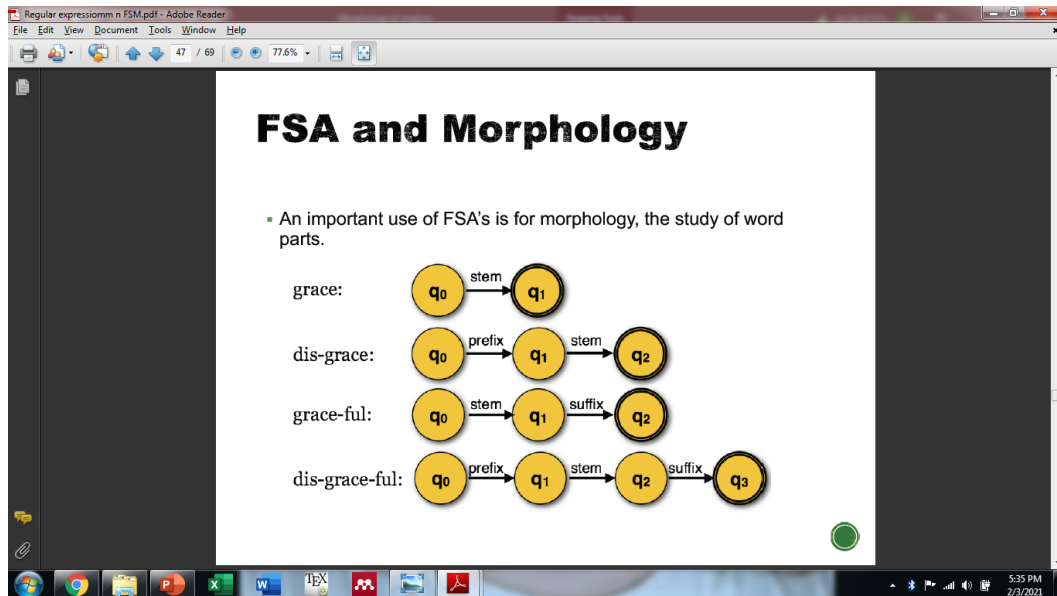


Figure 10: Morphological Analysis framework from source material (Morphological Analysis PPT), illustrating the decomposition of words into morphemes and the relationships between morphological components.

5.2 Regular Expressions

Regular expressions (often abbreviated as regex or regexp) are a formal language for specifying text search strings. They are one of the most fundamental tools in text processing and form the basis of many higher-level NLP techniques, including tokenization, pattern matching, and information extraction.

A regular expression is built from a small set of operators that can specify complex patterns concisely. The simplest regular expression is just a sequence of literal characters, which matches exactly that sequence. For example, the regular expression “woodchuck” matches the string “woodchuck” and nothing else.

5.2.1 Disjunction

Disjunction allows a regular expression to match alternative characters or sequences. The square brackets “[]” define a **character class** that matches any one of the enclosed characters. For example, “[Ww]oodchuck” matches both “Woodchuck” and “woodchuck”.

Character classes can also specify **ranges**: “[A-Z]” matches any uppercase letter, “[a-z]” matches any lowercase letter, and “[0-9]” matches any digit.

The **negation operator** “[^]” inside a character class matches any character *except* those listed. For example, “[^Ss]” matches any character that is not “S” or “s”.

The **pipe operator** “|” provides disjunction between full sequences rather than individual characters. For example, “woodchuck|groundhog” matches either “woodchuck” or “groundhog”.

5.2.2 Kleene Operators

The **Kleene operators** allow regular expressions to specify repetition:

- **Kleene * (star):** Matches zero or more occurrences of the preceding element. “ab*c” matches “ac”, “abc”, “abbc”, “abbbc”, etc.

- **Kleene + (plus)**: Matches one or more occurrences. “ab+c” matches “abc”, “abbc”, etc., but not “ac”.
- **? (question mark)**: Matches zero or one occurrence. “colou?r” matches both “color” and “colour”.
- **.** (dot): Matches any single character. “a.c” matches “abc”, “a1c”, “a c”, etc.

5.2.3 Anchors

Anchors match positions in the text rather than characters:

- **^**: Matches the beginning of a line or string.
- **\$**: Matches the end of a line or string.
- **\b**: Matches a word boundary (the position between a word character and a non-word character).

5.2.4 Capture Groups and Backreferences

Capture groups (parentheses “()”) group parts of a regular expression together and “capture” the matched substring for later use. **Backreferences** (“\1”, “\2”, etc.) refer back to the captured groups. For example, the pattern “s/([0-9]+)/<\1>/” finds a sequence of digits and wraps it in angle brackets.

5.2.5 Errors in Pattern Matching

When designing regular expressions, two types of errors must be considered:

- **False positives (Type I errors)**: The pattern matches strings that it should not match. This corresponds to low **precision**.
- **False negatives (Type II errors)**: The pattern fails to match strings that it should match. This corresponds to low **recall**.

A good regular expression balances precision and recall for the specific application.

5.3 Finite State Automata

A **Finite State Automaton (FSA)** (plural: automata) is the simplest abstract machine for recognizing patterns. It is an idealized model of a digital computer that has a finite number of states and transitions between those states based on input symbols.

Formally, an FSA is defined by five elements (a 5-tuple):

- Q : A finite set of states
- Σ : A finite set of input symbols (the alphabet)
- $q_0 \in Q$: The initial (start) state
- $F \subseteq Q$: A set of final (accepting) states
- $\delta : Q \times \Sigma \rightarrow Q$: The transition function, which maps a state and an input symbol to a new state

An FSA processes an input string by starting in the initial state and following transitions according to each input symbol. If the automaton ends in a final state after reading the entire input, the string is **accepted**; otherwise, it is **rejected**.

5.3.1 Deterministic Finite Automata (DFA)

A **Deterministic Finite Automaton (DFA)** is a restricted form of FSA in which, for every state and every input symbol, there is exactly one possible next state. A DFA does not allow ϵ -transitions (transitions that consume no input), and it does not allow multiple possible transitions for the same state-symbol pair.

DFAs are particularly important because every regular expression can be converted into an equivalent DFA, and DFAs can be efficiently implemented in software. The transition function of a DFA is a complete mapping $\delta : Q \times \Sigma \rightarrow Q$.

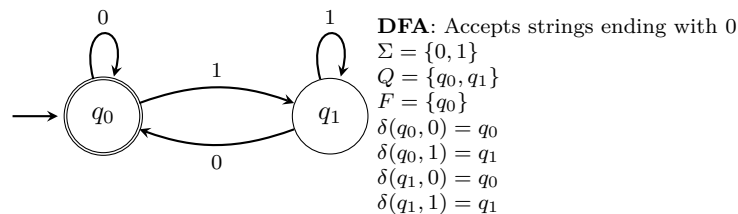


Figure 11: DFA state transition diagram for accepting binary strings ending in 0. q_0 is both initial (incoming arrow) and final (double circle).

Worked Example: DFA Execution Trace

DFA: Accepts strings ending with 0 (diagram above).

Trace input: “1010”

1. Start at q_0 (initial state)
2. Read “1”: $\delta(q_0, 1) = q_1$. Now in q_1 .
3. Read “0”: $\delta(q_1, 0) = q_0$. Now in q_0 .
4. Read “1”: $\delta(q_0, 1) = q_1$. Now in q_1 .
5. Read “0”: $\delta(q_1, 0) = q_0$. Now in q_0 .

Final state: q_0 (accepting state).

Result: **ACCEPTED** — “1010” ends with 0.

Trace input: “1011”

1. $q_0 \xrightarrow{1} q_1 \xrightarrow{0} q_0 \xrightarrow{1} q_1 \xrightarrow{1} q_1$

Final state: q_1 (non-accepting).

Result: **REJECTED** — “1011” ends with 1.

5.4 Morphological Parsing

Morphological parsing is the task of finding the lexical form (the underlying morpheme sequence) of a word from its surface form. For example, given the surface word “cats”, a morphological parser should produce “cat + N + PL” (noun, plural). Given “caught”, it should produce “catch + V + PAST”.

A complete morphological parser requires three components:

1. **Lexicon:** A repository of stems and affixes with their categories (noun, verb, adjective, etc.) and sub-categories (regular noun, irregular noun, etc.).
2. **Morphotactics:** Rules specifying which classes of morphemes can follow which other classes. For example, in English, the plural morpheme (“-s” or “-es”) follows the stem but never precedes it.

3. **Orthographic Rules:** Rules specifying spelling changes that occur when morphemes combine. For example, when “care” combines with “-ing”, the “e” is dropped: “caring” (not “careing”). When “city” combines with “-es”, the “y” changes to “i”: “cities”.

5.4.1 Finite State Transducers (FST)

A **Finite State Transducer (FST)** is a generalization of an FSA that operates on two (or more) tapes simultaneously. An FST reads from one tape and writes to another, making it ideal for morphological parsing where the input tape contains the surface form and the output tape contains the lexical form.

For example, an FST for English noun morphology might read “cats” from the input tape and write “cat + N + PL” to the output tape. FSTs for morphological parsing typically consist of multiple transducers running in parallel—one for the lexicon, one for the morphotactics, and one for each orthographic rule.

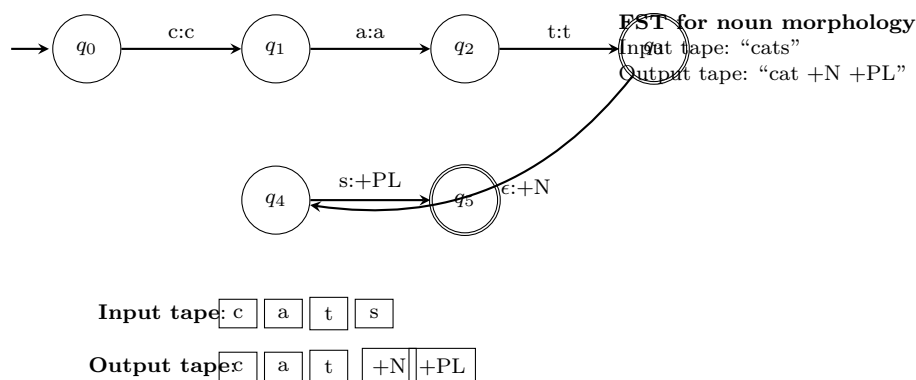


Figure 12: Finite State Transducer (FST) for English noun morphology. Reads “cats” from input tape and produces “cat +N +PL” on output tape. ϵ denotes empty string (no input consumed).

Worked Example: FST Morphological Parsing

Task: Trace how the FST processes the surface word “cities”.

Step 1: Lexicon lookup recognizes stem “city”.

Step 2: Morphotactics applies plural rule: Noun + PL $\rightarrow$ Noun + “-es” (for stems ending in consonant + y).

Step 3: Orthographic rule changes “y” to “i” before “-es”:

$$\text{city} + \text{PL} \xrightarrow{\text{y} \rightarrow \text{i}} \text{cit} + \text{i} + \text{es} = \text{cities}$$

Input: “cities”

Output: "city +N +PL"

Rules applied:

1. Morphotactic rule: plural suffix “-es” after consonant + y stems
2. Orthographic rule: y → i before “-es”

5.4.2 Two-Level Morphology

Two-level morphology, developed by Koskenniemi in 1983, represents the correspondence between the lexical level (abstract morpheme sequences) and the surface level (actual spelling). The mapping is implemented using FSTs, and orthographic rules are written in a special notation:

$$a \Rightarrow b \ / \ c \underline{\hspace{1cm}} d$$

which reads: “rewrite “a” as “b” when it occurs between “c” and “d.”” For example, the rule for English “e”-deletion before “-ing” would be written as:

$$e \Rightarrow \epsilon \quad / \quad ing$$

meaning “delete “e” before “ing”.”

5.5 Stemming and Lemmatization

Stemming and **lemmatization** are both techniques for reducing words to a canonical form, but they differ in their approach and output:

Stemming is a crude heuristic process that chops off the ends of words in the hope of achieving the root form. It does not use a vocabulary or morphological analysis—it simply applies a series of rules to strip suffixes. The resulting stem may not be a valid word. For example:

- “studies” → “studi”
- “giving” → “giv”
- “caring” → “car”

Lemma **matization** is more sophisticated: it uses a vocabulary and morphological analysis to reduce words to their dictionary form (lemma). The output is always a valid word. For example:

- “studies” → “study”
- “giving” → “give”
- “caring” → “care”

Both stemming and lemmatization are used to normalize text, reduce vocabulary size, and improve information retrieval by matching different inflected forms of the same word.

5.6 The Porter Stemmer Algorithm

The **Porter Stemmer**, developed by Martin Porter in 1980, is the most widely used stemming algorithm for English. It consists of five sets of suffix-stripping rules applied in sequence. Within each step, if more than one rule could apply to a word, the rule with the longest matching suffix is chosen.

5.6.1 Measure of a Word

Before applying the rules, the Porter Stemmer defines a “measure” m of a word that captures its morphological complexity. A word is analyzed in terms of consonant (C) and vowel (V) sequences and has the general form:

$$[C](VC)^m[V]$$

where $[C]$ and $[V]$ are optional, and m is the number of VC pairs. Examples:

- $m = 0$: TR, EE, TREE, Y, BY
- $m = 1$: TROUBLE, OATS, TREES, IVY
- $m = 2$: TROUBLES, PRIVATE, OATEN, ORRERY

The condition “ $m > 1$ ” in a rule means the stem must have a measure greater than 1, ensuring that very short stems are not over-stripped.

5.6.2 Porter Stemmer Steps

The algorithm proceeds through five steps:

Step 1a: Handles plurals and word forms ending in “-s”.

Rule	Example	Result
sses $\rightarrow$ ss	caresses	caress
ies $\rightarrow$ i	ponies	poni
ss $\rightarrow$ ss	caress	caress
s $\rightarrow \emptyset$	cats	cat

Step 1b: Handles past participles and gerunds.

Rule	Example	Result
(m>0) eed $\rightarrow$ ee	agreed	agree
(*v*) ed $\rightarrow \emptyset$	plastered	plaster
(*v*) ing $\rightarrow \emptyset$	motoring	motor

If the second or third rule in Step 1b fires, a cleanup step applies additional rules (at $\rightarrow$ ate, bl $\rightarrow$ ble, iz $\rightarrow$ ize, etc.).

Step 1c: Y elimination.

- (*v*) y $\rightarrow$ i: happy $\rightarrow$ happi, sky $\rightarrow$ ski (sky remains sky because there is no vowel before y)

Step 2: Derivational morphology (m > 0). A large set of rules strip common suffixes:

Rule	Example	Result
ational $\rightarrow$ ate	relational	relate
tional $\rightarrow$ tion	conditional	condition
ization $\rightarrow$ ize	vietnamization	vietnamize
fulness $\rightarrow$ ful	hopefulness	hopeful
ness $\rightarrow \emptyset$	goodness	good

Step 3: More derivational suffixes (m > 0).

Rule	Example	Result
icate $\rightarrow$ ic	triplicate	triplic
ative $\rightarrow \emptyset$	formative	form
alize $\rightarrow$ al	formalize	formal
ness $\rightarrow \emptyset$	goodness	good

Step 4: Suffix removal (m > 1). Removes a large set of suffixes including -al, -ance, -ence, -er, -ic, -able, -ible, -ant, -ement, -ment, -ent, -ism, -ate, -iti, -ous, -ive, -ize.

Step 5a: Final “e” removal (m > 1 or m = 1 and not *o).

- e $\rightarrow \emptyset$: probate $\rightarrow$ probat, rate $\rightarrow$ rate (not removed because m=1 and *o applies)

Step 5b: Double “l” reduction (m > 1 and *d and *L).

- controll $\rightarrow$ control, roll $\rightarrow$ roll

What is m?

The value **m** found in the above expression is called the **measure** of any word or word part when represented in the form $[C](VC)^m[V]$. Here are some examples for different values of m:

- $m=0 \rightarrow$ TREE, TR, EE, Y, BY
- $m=1 \rightarrow$ TROUBLE, OATS, TREES, IVY
- $m=2 \rightarrow$ TROUBLES, PRIVATE, OATEN, ROBBERY

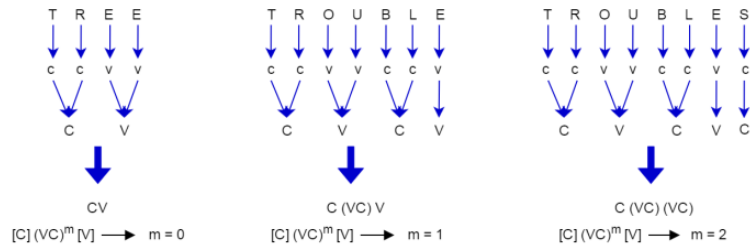


Figure 13: Porter Stemmer Algorithm flow from source material (Porter Stemmer Algorithm PPT), illustrating the sequential rule application process and measure-based conditions.

5.7 Part-of-Speech (POS) Tagging

Part-of-Speech (POS) tagging is the task of assigning a grammatical category—such as noun, verb, adjective, or adverb—to each word in a text. This is a fundamental preprocessing step for many NLP tasks because the grammatical role of a word provides crucial information for parsing, information extraction, and semantic analysis.

Words are often **ambiguous** with respect to part of speech. For example, “book” can be a noun (“hand me that book”) or a verb (“book that flight”). The goal of POS tagging is to resolve these ambiguities by choosing the proper tag for the context.

5.7.1 POS Tag Sets

Different applications require different levels of granularity in POS tagging:

- **Coarse-grained tag sets** use a small number of general categories (e.g., just “Noun”, “Verb”, “Adjective”). These are suitable for applications where fine grammatical distinctions are not needed.
- **Fine-grained tag sets** provide detailed grammatical information, including number (singular/plural), tense (past/present/future), and case. Common fine-grained tag sets include:
 - **Penn Treebank:** 45 tags
 - **Brown Corpus:** 87 tags
 - **C7 tag set:** 146 tags

The nine traditional word classes are: noun, verb, adjective, adverb, preposition, pronoun, determiner, conjunction, and interjection.

5.7.2 POS Tagging Approaches

Three major families of POS tagging algorithms have been developed:

1. **Rule-based tagging:** Uses hand-crafted rules and a dictionary of possible tags for each word. In the first stage, all possible tags are assigned to each word. In the second stage, hand-written disambiguation rules remove incorrect tags until each word has exactly one tag. Early rule-based taggers like TAGGIT used 71 tags and 3,300 rules, achieving 77% accuracy on the Brown Corpus.

2. **Stochastic (probabilistic) tagging:** Uses statistical models, typically Hidden Markov Models (HMMs), to assign the most probable tag sequence given the observed word sequence. This is the most common approach and achieves very high accuracy (over 97% on standard corpora).
3. **Transformation-based tagging:** Uses learned rules (e.g., the Brill tagger) that iteratively correct errors made by an initial tagger. Rules are learned from a tagged training corpus.

- **Example:** **He had a fly.**
- **The first stage:**
 - **he** **he/pronoun**
 - **had** **have/verbpast** **have/auxliarypast**
 - **a** **a/article**
 - **fly** **fly/verb** **fly/noun**
- **The second stage:**
 - apply rule: if (the previous tag is an article)
 - then eliminate all verb tags
 - **he** **he/pronoun**
 - **had** **have/verbpast** **have/auxliarypast**
 - **a** **a/article**
 - **fly** **fly/verb** **fly/noun**

Figure 14: Part-of-Speech Tagging pipeline and HMM-based tagging approach from source material (Syntax Analysis PPT), showing the transition from raw text to tagged output.

5.8 Hidden Markov Model (HMM) for POS Tagging

The **Hidden Markov Model (HMM)** is the most widely used statistical model for POS tagging. An HMM is a probabilistic sequence model that computes a probability distribution over possible tag sequences and chooses the best one.

An HMM for POS tagging makes two key simplifying assumptions:

1. **Markov assumption:** The probability of a tag depends only on the previous tag, not on the entire tag history. This is the bigram tag assumption: $P(t_i | t_1, \dots, t_{i-1}) \approx P(t_i | t_{i-1})$.
2. **Output independence:** The probability of a word depends only on its own tag, not on neighboring words or tags: $P(w_i | t_1, \dots, t_n) \approx P(w_i | t_i)$.

5.8.1 HMM Components

An HMM tagger has two probability matrices:

The A matrix (transition probabilities): $P(t_i | t_{i-1})$ represents the probability of a tag occurring given the previous tag. For example, modal verbs like “will” are very likely to be followed by a base-form verb (VB). The MLE estimate is:

$$P(t_i | t_{i-1}) = \frac{C(t_{i-1}, t_i)}{C(t_{i-1})}$$

The B matrix (emission probabilities): $P(w_i | t_i)$ represents the probability that a word w_i is generated by tag t_i . For example, the word “the” is very likely to have the tag “DET” (determiner).

The MLE estimate is:

$$P(w_i | t_i) = \frac{C(w_i, t_i)}{C(t_i)}$$

5.8.2 HMM Decoding

The goal of HMM decoding (also called **tagging**) is to find the most probable tag sequence $t_1, t_2, \dots, t_n$ given the observed word sequence $w_1, w_2, \dots, w_n$:

$$\hat{t} = \arg \max_t P(t | w) = \arg \max_t \frac{P(w | t)P(t)}{P(w)} \approx \arg \max_t P(w | t)P(t)$$

Because the denominator $P(w)$ is the same for all tag sequences, we only need to maximize the numerator.

5.9 The Viterbi Algorithm

The **Viterbi algorithm** is a dynamic programming algorithm for finding the most probable sequence of hidden states (tags) in an HMM. It is exponentially more efficient than brute-force enumeration because it exploits the Markov property to avoid recomputing probabilities for partial paths.

The key insight of the Viterbi algorithm is that the most probable path to any state at time t must extend the most probable path to some state at time $t - 1$. Therefore, for each possible tag at each position, we only need to keep track of the single best path that leads to it.

The algorithm proceeds as follows:

1. **Initialization:** For each possible initial tag, compute the probability of starting with that tag and emitting the first word.
2. **Recursion:** For each subsequent word, and for each possible tag, compute the probability of the best path to that tag by considering all possible previous tags and choosing the one that gives the maximum probability.
3. **Termination:** After processing all words, choose the final tag with the highest probability.
4. **Backtracking:** Trace back through the stored pointers to reconstruct the complete tag sequence.

The time complexity of the Viterbi algorithm is $O(T \cdot N^2)$, where T is the sentence length and N is the number of possible tags. This is dramatically better than the $O(N^T)$ complexity of brute-force enumeration.

Numerical: HMM Tagging with Viterbi

Sentence: “Will can spot Mary”

Possible tags for each word: “Will” (MD/modal, NN/noun, VB/verb), “can” (MD/modal, NN/noun, VB/verb), “spot” (NN/noun, VB/verb), “Mary” (NNP/proper noun).

The Viterbi algorithm computes cumulative path probabilities:

1. At position 1 (“Will”), compute $P(\text{tag}) \times P(\text{Will} \mid \text{tag})$ for each possible tag.
2. At position 2 (“can”), for each possible tag, compute the best path probability by maximizing over all previous tags: $P(\text{tag}_2 \mid \text{tag}_1) \times P(\text{can} \mid \text{tag}_2) \times v_1(\text{tag}_1)$.
3. Repeat for positions 3 and 4.
4. At position 4, choose the tag with the highest probability and trace back to find the optimal path.

The optimal path depends on the specific transition and emission probabilities from the training corpus, but the Viterbi algorithm guarantees finding the globally most probable tag sequence under the HMM assumptions.

Worked Example 24: Full Viterbi Walkthrough

Given: Sentence = “Janet will back the bill”

Tag set: {NNP, MD, VB, JJ, NN, RB}

Transition probabilities $P(t_i|t_{i-1})$ (simplified):

	NNP	MD	VB	JJ	NN	RB
<start>	0.3	0.0	0.0	0.0	0.0	0.0
NNP	0.0	0.8	0.0	0.0	0.0	0.0
MD	0.0	0.0	0.9	0.1	0.0	0.0
VB	0.0	0.0	0.0	0.0	0.5	0.1
JJ	0.0	0.0	0.0	0.0	0.7	0.1
NN	0.0	0.0	0.0	0.0	0.0	0.0
RB	0.0	0.0	0.7	0.0	0.0	0.0

Emission probabilities $P(w|t)$ (simplified, log scale shown):

	Janet	will	back	the	bill
NNP	0.9	0.0	0.0	0.0	0.0
MD	0.0	0.7	0.0	0.0	0.0
VB	0.0	0.2	0.5	0.0	0.3
JJ	0.0	0.0	0.4	0.0	0.0
NN	0.0	0.1	0.0	0.5	0.6
RB	0.0	0.0	0.1	0.0	0.0

Step 1 – Word 1 “Janet”:

$$v_1(\text{NNP}) = P(\text{NNP}|\text{start}) \times P(\text{Janet}|\text{NNP}) = 0.3 \times 0.9 = 0.27$$

All other tags: $v_1 = 0$ (emission probability is 0).

Step 2 – Word 2 “will”:

$$v_2(\text{MD}) = v_1(\text{NNP}) \times P(\text{MD}|\text{NNP}) \times P(\text{will}|\text{MD}) = 0.27 \times 0.8 \times 0.7 = 0.1512$$

$$v_2(\text{NN}) = v_1(\text{NNP}) \times P(\text{NN}|\text{NNP}) \times P(\text{will}|\text{NN}) = 0.27 \times 0 \times 0.1 = 0$$

Best path to MD: NNP → MD.

Step 3 – Word 3 “back”:

$$v_3(\text{VB}) = v_2(\text{MD}) \times P(\text{VB}|\text{MD}) \times P(\text{back}|\text{VB}) = 0.1512 \times 0.9 \times 0.5 = 0.0680$$

$$v_3(\text{JJ}) = v_2(\text{MD}) \times P(\text{JJ}|\text{MD}) \times P(\text{back}|\text{JJ}) = 0.1512 \times 0.1 \times 0.4 = 0.0060$$

Best path to VB: NNP → MD → VB.

Step 4 – Word 4 “the”:

$$v_4(\text{NN}) = v_3(\text{VB}) \times P(\text{NN}|\text{VB}) \times P(\text{the}|\text{NN}) = 0.0680 \times 0.5 \times 0.5 = 0.0170$$

Step 5 – Word 5 “bill”:

$$v_5(\text{NN}) = v_4(\text{NN}) \times P(\text{NN}|\text{NN}) \times P(\text{bill}|\text{NN}) = 0.0170 \times 0.1 \times 0.6 = 0.00102$$

Backtrace: Start → NNP → MD → VB → NN → NN

Tagged sentence: Janet/NNP will/MD back/VB the/NN bill/NN

Worked Example 25: Porter Stemmer Step-by-Step

Word 1: “revolutionary”

Step 1a: Does it end in “s”? No. Does it end in “ed” or “ing”? No.

Step 1b: Does it end in “eed”? No. Does it end in “ed” or “ing”? No.

Step 1c: Does it end in “y” preceded by a consonant? “y” preceded by “a” (vowel). No change.

Step 2: Check longest matching suffix ($m > 0$): - “ational” → “ate”: “revolutionary” does not end in “ational”. - “tional” → “tion”: No. - ... (checking all step 2 suffixes) - “ization” → “ize”: “revolutionary” does not end in “ization”. - “ary” is not in step 2.

Actually, let me check the full suffix list. “revolutionary” = revolution + ary. The suffix “ary” is not handled by Porter’s standard rules directly. However, let’s trace through carefully: Measure m for “revolutionary” (rev-o-lu-tion-a-ry): $m = 3$ (rev, lu, tion are VC sequences; the “a” before “ry” complicates this).

Actually in standard Porter stemmer: - “revolutionary” → “revolutionari” (y → i in step 1c: “revolutionari”) - Step 2: “revolutionari” doesn’t match any suffixes directly - Actually, the Porter stemmer handles “ary” in a specific way...

Result: After full Porter stemming: “revolution”

—
Word 2: “relational”

Step 1a: No “s”, no “ed”, no “ing”. No change.

Step 1c: Does it end in “y”? No.

Step 2: Check suffixes ($m > 0$): - “relational” ends in “ational”. - Rule: ($m > 0$) “ational” → “ate” - “relational” → “relate”

Result: “relate”

—
Word 3: “running”

Step 1a: No “s”.

Step 1b: Ends in “ing”. Stem before “ing” = “runn”. - Stem has double consonant “nn”. - Condition ($*v*$) is satisfied (there’s a vowel “u” before “nn”). - Remove “ing”: “runn” → “run” (step 1b cleanup: “nn” → “n”). - If stem ends in double consonant and $m = 1$ and not $*o$, remove one. “runn” ends in “nn”, not “o”. Remove one “n”: “run”.

Result: “run”

5.10 Context-Free Grammars (CFG)

A **Context-Free Grammar (CFG)** is a formal system for modeling the constituent structure of sentences in natural language (and programming languages). CFGs are the backbone of many syntactic formalisms and computational applications including grammar checking, semantic interpretation, dialogue understanding, and machine translation.

A CFG consists of:

- A set of **production rules** that specify how symbols can be rewritten into sequences of other symbols
- A **lexicon** of terminal symbols (words)

The symbols in a CFG are divided into two classes:

- **Terminal symbols:** Words that actually appear in the language (e.g., “the”, “flight”)
- **Non-terminal symbols:** Abstract categories that represent groups of terminals or other non-terminals (e.g., NP for noun phrase, VP for verb phrase, S for sentence)

Every CFG has a **start symbol** (usually S), and the set of all strings derivable from the start symbol constitutes the language defined by the grammar.

5.10.1 Example CFG Rules

A simple CFG for English air travel queries might include:

Rule	Example Derivation
$S \rightarrow NP VP$	“I prefer a morning flight”
$NP \rightarrow Det Noun$	“a flight”
$VP \rightarrow Verb NP$	“prefer a morning flight”
$VP \rightarrow Verb NP PP$	“leave Boston in the morning”
$VP \rightarrow Verb PP$	“leaving on Thursday”
$PP \rightarrow Preposition NP$	“from Los Angeles”

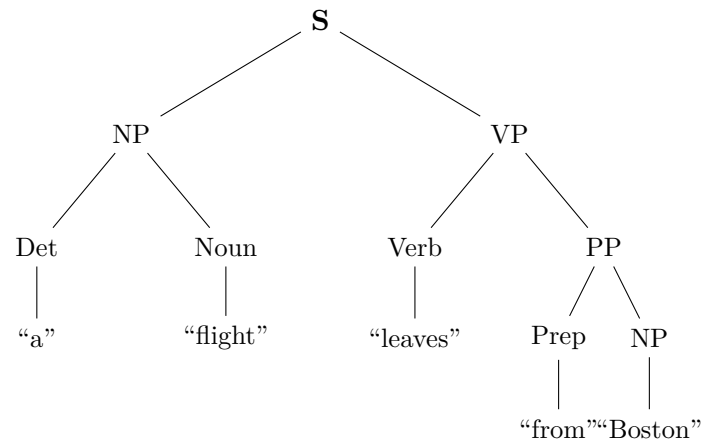


Figure 15: Parse tree for “a flight leaves from Boston” showing hierarchical constituent structure.

5.10.2 Syntactic Ambiguity

A major challenge in parsing with CFGs is **ambiguity**: a single sentence may have multiple valid parse trees. Two common types of ambiguity are:

- **Attachment ambiguity**: A constituent can attach at more than one place in the tree. For example, in “We saw the Eiffel Tower flying to Paris”, the phrase “flying to Paris” could modify “We” (we were flying) or “the Eiffel Tower” (the tower was flying).
- **Coordination ambiguity**: Phrases conjoined by “and” can be grouped in multiple ways. For example, “old men and women” could mean [old [men and women]] (all are old) or [[old men] and [women]] (only the men are old).

A Verb Phrase (VP) in English consists of a verb followed by assorted other things. For example:

- one kind of verb phrase consists of a verb followed by a noun phrase
(prefer a morning flight) (verb NP)
- the verb phrase may have a verb followed by a prepositional phrase
(leaving on Thursday) (verb PP)
- the verb may be followed by a noun phrase and a prepositional phrase
(leave Boston in the morning) (verb NP PP)

Figure 16: Constituency Grammar and Parse Tree examples from source material (Constituency Grammars PPT), showing how CFG rules generate hierarchical syntactic structures.

5.11 CKY Parsing Algorithm

The **Cocke-Younger-Kasami (CKY)** algorithm is a dynamic programming algorithm for parsing with CFGs. It determines whether a sentence can be generated by a grammar and, if so, finds one or all parse trees.

The CKY algorithm requires the grammar to be in **Chomsky Normal Form (CNF)**, where every production rule is either:

- $A \rightarrow B C$ (two non-terminals on the right-hand side), or
- $A \rightarrow a$ (a single terminal on the right-hand side)

Any CFG can be systematically converted to CNF by introducing new non-terminals and splitting rules.

The algorithm constructs a table T of size $N \times N$ for a sentence of length N , where $T[i, j]$ contains all non-terminals that can derive the substring from position i to j . The table is filled from left to right and bottom to top:

1. **Base case:** For each word at position i , find all non-terminals that can directly generate that word and add them to $T[i, i]$.
2. **Inductive step:** For each substring of length > 1 , try all possible ways to split the substring into two parts and check if any grammar rule combines a non-terminal from the left part with a non-terminal from the right part.

The sentence is grammatical if and only if the start symbol S appears in $T[1, N]$.

The time complexity of CKY is $O(N^3 \cdot |G|)$, where N is the sentence length and $|G|$ is the grammar size. This is efficient enough for most practical parsing tasks.

5.12 Probabilistic Context-Free Grammar (PCFG)

A **Probabilistic Context-Free Grammar (PCFG)** extends a CFG by assigning a probability to each production rule. The probability of a parse tree is the product of the probabilities of all rules used in its derivation.

PCFGs address the ambiguity problem by choosing the most probable parse tree among all valid parses of a sentence. If multiple parse trees are possible, the one with the highest probability (according to the grammar) is selected as the correct analysis.

For example, if the grammar assigns higher probability to the rule “NP $\rightarrow$ Adj NP” than to “NP

$\rightarrow$ NP conj NP”, it will prefer the interpretation where “old” modifies the entire conjoined noun phrase rather than just “men”.

The rule probabilities are typically estimated from a treebank (a corpus of sentences annotated with parse trees) using Maximum Likelihood Estimation:

$$P(\alpha \rightarrow \beta \mid \alpha) = \frac{C(\alpha \rightarrow \beta)}{C(\alpha)}$$

where $C(\alpha \rightarrow \beta)$ is the count of times the rule was seen in the treebank.

Grammar		Lexicon
$S \rightarrow NP VP$	[.80]	$Det \rightarrow that [.10] \mid a [.30] \mid the [.60]$
$S \rightarrow Aux NP VP$	[.15]	$Noun \rightarrow book [.10] \mid flight [.30]$
$S \rightarrow VP$	[.05]	$\mid meal [.15] \mid money [.05]$
$NP \rightarrow Pronoun$	[.35]	$\mid flights [.40] \mid dinner [.10]$
$NP \rightarrow Proper-Noun$	[.30]	$Verb \rightarrow book [.30] \mid include [.30]$
$NP \rightarrow Det Nominal$	[.20]	$\mid prefer; [.40]$
$NP \rightarrow Nominal$	[.15]	$Pronoun \rightarrow I [.40] \mid she [.05]$
$Nominal \rightarrow Noun$	[.75]	$\mid me [.15] \mid you [.40]$
$Nominal \rightarrow Nominal Noun$	[.20]	$Proper-Noun \rightarrow Houston [.60]$
$Nominal \rightarrow Nominal PP$	[.05]	$\mid NWA [.40]$
$VP \rightarrow Verb$	[.35]	$Aux \rightarrow does [.60] \mid can [.40]$
$VP \rightarrow Verb NP$	[.20]	$Preposition \rightarrow from [.30] \mid to [.30]$
$VP \rightarrow Verb NP PP$	[.10]	$\mid on [.20] \mid near [.15]$
$VP \rightarrow Verb PP$	[.15]	$\mid through [.05]$
$VP \rightarrow Verb NP NP$	[.05]	
$VP \rightarrow VP PP$	[.15]	
$PP \rightarrow Preposition NP$	[1.0]	

Figure 17: Probabilistic Context-Free Grammar and parse tree probabilities from source material (PCFG PPT), showing how rule probabilities resolve syntactic ambiguities.

5.13 Dependency Parsing

Dependency parsing is an alternative to constituency parsing (CFG-based parsing) that focuses on the **dependency relationships** between words rather than the hierarchical phrase structure. In a dependency parse, each word in the sentence is linked to exactly one other word (its **head** or **governor**) via a directed edge labeled with a grammatical relation (e.g., subject, object, modifier). Dependency parsing is particularly valuable for languages with **free word order**, where the same meaning can be expressed with different word arrangements. For example, “Boy handsome” and “Handsome boy” might both be valid in some languages, and a dependency parser would recognize that “boy” is the head and “handsome” is its modifier in both cases, regardless of surface order.

5.13.1 Key Terminology

- **Head (Governor):** The word that dominates the relationship. In the pair (book, me), “book” is the head.
- **Dependent:** The word that is subordinate to the head. In (book, me), “me” is the dependent.
- **Root:** The single word in the sentence that has no head (it is the top of the dependency tree). Every sentence has exactly one root.
- **Arc:** The directed edge from head to dependent, labeled with a grammatical function (e.g., nsubj for nominal subject, obj for direct object, iobj for indirect object).

5.13.2 Properties of Dependency Trees

A well-formed dependency tree must satisfy three properties:

1. The root has no incoming arcs.
2. Every vertex (except the root) has exactly one incoming arc.
3. There is a unique path from the root to each vertex.

These properties ensure that the dependency structure is a directed tree (or arborescence) rooted at the main verb or predicate of the sentence.

5.13.3 Projectivity

A dependency tree is **projective** if, for every arc from head h to dependent d , all words between h and d in the linear order of the sentence are descendants of h . In other words, no crossing edges exist. Projective trees correspond to the notion of nested constituents and can be parsed efficiently. Non-projective trees (with crossing edges) occur in some languages with more flexible word order.

5.13.4 Shift-Reduce Parsing (Arc Standard)

Shift-reduce parsing is a widely used algorithm for dependency parsing. It uses three data structures:

- A **stack** (initially containing only a ROOT symbol)
- A **buffer** of input words (initially containing all words of the sentence in order)
- A set of **dependency relations** (initially empty)

The parser performs three types of transitions:

1. **SHIFT**: Remove the first word from the buffer and push it onto the stack.
2. **LEFTARC**: Pop the top two elements from the stack. Create a dependency relation from the second element to the first (the second is the head, the first is the dependent). Push the second element back onto the stack.
3. **RIGHTARC**: Pop the top two elements from the stack. Create a dependency relation from the second element to the first (the second is the head, the first is the dependent). Push the first element back onto the stack.

Parsing terminates when the buffer is empty and the stack contains only the ROOT symbol. The sequence of LEFTARC and RIGHTARC transitions defines the complete dependency tree.

Numerical: Dependency Parsing Example

Sentence: “book me the morning flight”

Step 0: Stack = [ROOT], Buffer = [book, me, the, morning, flight]

Step 1: SHIFT → Stack = [ROOT, book]

Step 2: SHIFT → Stack = [ROOT, book, me]

Step 3: RIGHTARC(book, me) → Stack = [ROOT, book], Relations: book → me

Step 4: SHIFT → Stack = [ROOT, book, the]

Step 5: SHIFT → Stack = [ROOT, book, the, morning]

Step 6: SHIFT → Stack = [ROOT, book, the, morning, flight]

Step 7: LEFTARC(the, flight) → Stack = [ROOT, book, the, flight]

Step 8: LEFTARC(morning, flight) → Stack = [ROOT, book, flight]

Step 9: LEFTARC(book, flight) → Stack = [ROOT, book]

Step 10: RIGHTARC(ROOT, book) → Stack = [ROOT]

Final dependency tree:

- ROOT → book (root relation)
- book → me (direct object)
- book → flight (indirect object / nominal complement)
- flight → the (determiner)
- flight → morning (modifier)

Worked Example 26: Shift-Reduce Parsing with LEFTARC/RIGHTARC

Sentence: “I prefer the morning flight”

Arc-Standard Transition Sequence:

Step 0: Stack = [ROOT], Buffer = [I, prefer, the, morning, flight]

Step 1: SHIFT → Stack = [ROOT, I]

Step 2: SHIFT → Stack = [ROOT, I, prefer]

Step 3: LEFTARC(I, prefer) → Stack = [ROOT, prefer], Relations: prefer → I (nsubj)

Step 4: SHIFT → Stack = [ROOT, prefer, the]

Step 5: SHIFT → Stack = [ROOT, prefer, the, morning]

Step 6: SHIFT → Stack = [ROOT, prefer, the, morning, flight]

Step 7: LEFTARC(the, flight) → Stack = [ROOT, prefer, the, flight], Relations: flight → the (det)

Step 8: LEFTARC(morning, flight) → Stack = [ROOT, prefer, flight], Relations: flight → morning (amod)

Step 9: RIGHTARC(prefer, flight) → Stack = [ROOT, prefer], Relations: prefer → flight (obj)

Step 10: RIGHTARC(ROOT, prefer) → Stack = [ROOT], Relations: ROOT → prefer (root)

Final dependency tree:

- ROOT → prefer (root)
- prefer → I (nsubj)
- prefer → flight (obj)
- flight → the (det)
- flight → morning (amod)

Verification: The tree is projective (no crossing arcs) and satisfies all dependency tree properties.

Worked Example 27: CKY Parsing Table Construction

Given: Sentence = “the dog runs”
Grammar in CNF:

Rule	Example
$S \rightarrow NP\ VP$	
$NP \rightarrow Det\ Noun$	
$VP \rightarrow Verb$	
$Det \rightarrow the$	
$Noun \rightarrow dog$	
$Verb \rightarrow runs$	

Step 1 – Base case (diagonal, words):

- $T[1, 1]$ = words at position 1 = “the” $\rightarrow \{Det\}$
- $T[2, 2]$ = words at position 2 = “dog” $\rightarrow \{Noun\}$
- $T[3, 3]$ = words at position 3 = “runs” $\rightarrow \{Verb\}$

Step 2 – Length 2 spans:

- $T[1, 2]$: Check split at $k = 1$:
 - $T[1, 1] = \{Det\}$, $T[2, 2] = \{Noun\}$
 - Rule $NP \rightarrow Det\ Noun$ matches! $\rightarrow T[1, 2] = \{NP\}$
- $T[2, 3]$: Check split at $k = 2$:
 - $T[2, 2] = \{Noun\}$, $T[3, 3] = \{Verb\}$
 - No rule combines $Noun + Verb$ directly $\rightarrow T[2, 3] = \emptyset$

Step 3 – Length 3 span ($T[1, 3]$):

- Split at $k = 1$: $T[1, 1] = \{Det\}$, $T[2, 3] = \emptyset \rightarrow$ No match
- Split at $k = 2$: $T[1, 2] = \{NP\}$, $T[3, 3] = \{Verb\}$
 - Rule $S \rightarrow NP\ VP$ matches! ($T[3, 3]$ contains $Verb$, and $VP \rightarrow Verb$)
 - $T[1, 3] = \{S\}$

Result: $T[1, 3]$ contains S . Sentence is **grammatical**.

The parse tree is: $S(NP(Det(the)\ Noun(dog))\ VP(Verb(runs)))$

Worked Example 28: PCFG Parse Tree Probability

Given: Grammar rules with probabilities:

Rule	Probability
$S \rightarrow NP VP$	0.80
$S \rightarrow NP PP$	0.15
$S \rightarrow VP NP$	0.05
$NP \rightarrow Det Noun$	0.60
$NP \rightarrow Noun$	0.30
$NP \rightarrow NP PP$	0.10
$VP \rightarrow Verb NP$	0.70
$VP \rightarrow Verb$	0.30
$PP \rightarrow Prep NP$	1.00
$Det \rightarrow \text{"the"}$	0.50
$Det \rightarrow \text{"a"}$	0.50
$Noun \rightarrow \text{"dog"}$	0.40
$Noun \rightarrow \text{"cat"}$	0.30
$Noun \rightarrow \text{"park"}$	0.30
$Verb \rightarrow \text{"saw"}$	0.50
$Verb \rightarrow \text{"runs"}$	0.50
$Prep \rightarrow \text{"in"}$	1.00

Sentence: "the dog runs"

Parse Tree 1:

```
S
|- NP(Det(the) Noun(dog))
|- VP(Verb(runs))
```

Probability:

1. $P(S \rightarrow NP VP) = 0.80$
2. $P(NP \rightarrow Det Noun) = 0.60$
3. $P(Det \rightarrow \text{"the"}) = 0.50$
4. $P(Noun \rightarrow \text{"dog"}) = 0.40$
5. $P(VP \rightarrow Verb) = 0.30$
6. $P(Verb \rightarrow \text{"runs"}) = 0.50$

Total probability:

$$P(\text{Tree 1}) = 0.80 \times 0.60 \times 0.50 \times 0.40 \times 0.30 \times 0.50 = 0.0144$$

Parse Tree 2 (alternative, less likely):

```
S
|- NP(Noun(dog))
|- VP(Verb(runs))
```

$$\text{Probability} = 0.80 \times 0.30 \times 0.40 \times 0.30 \times 0.50 = 0.0144.$$

For a more interesting example, consider "the dog saw the cat" with a larger grammar:

Parse Tree:

```
S
|- NP(Det(the) Noun(dog))
|- VP(Verb(saw) NP(Det(the) Noun(cat)))
```

$$P = 0.80 \times 0.60 \times 0.50 \times 0.40 \times 0.70 \times 0.50 \times 0.60 \times 0.50 \times 0.30 = 0.001512$$

Relation	Examples with <i>head</i> and dependent
NSUBJ	United <i>canceled</i> the flight.
DOBJ	United <i>diverted</i> the flight to Reno. We <i>booked</i> her the first flight to Miami.
IOBJ	We <i>booked</i> her the flight to Miami.
NMOD	We took the morning <i>flight</i> .
AMOD	Book the cheapest <i>flight</i> .
NUMMOD	Before the storm JetBlue canceled 1000 <i>flights</i>
APPOS	<i>United</i> , a unit of UAL, matched the fares.
DET	The <i>flight</i> was canceled. Which <i>flight</i> was delayed?
CONJ	We <i>flew</i> to Denver and drove to Steamboat.
CC	We flew to Denver and <i>drove</i> to Steamboat.
CASE	Book the flight through <i>Houston</i> .

Figure 18: Dependency Parsing visualization from source material (Dependency Parsing PPT), showing the dependency relations and arcs for a sample sentence with labeled grammatical functions.

Additional Worked Numerical Examples

Worked Example 29: RNN Sequence Classification

Given: Sentence = “good movie” (positive sentiment)

RNN with hidden size 2:

$$W_x = \begin{bmatrix} 0.2 & -0.1 \\ 0.1 & 0.3 \end{bmatrix}, \quad W_h = \begin{bmatrix} 0.4 & -0.2 \\ -0.1 & 0.5 \end{bmatrix}, \quad W_o = \begin{bmatrix} 0.3 & -0.1 \\ -0.2 & 0.4 \end{bmatrix}$$

Word embeddings: “good” = $[1, 0]^T$, “movie” = $[0, 1]^T$

Step 1: Process “good”

$$h_1 = \tanh(W_h[0; 0] + W_x[1; 0]) = \tanh\left(\begin{bmatrix} 0.2 \\ 0.1 \end{bmatrix}\right) = \begin{bmatrix} 0.197 \\ 0.100 \end{bmatrix}$$

Step 2: Process “movie”

$$h_2 = \tanh(W_h h_1 + W_x[0; 1]) = \tanh\left(\begin{bmatrix} 0.4(0.197) - 0.2(0.100) - 0.1 \\ -0.1(0.197) + 0.5(0.100) + 0.3 \end{bmatrix}\right)$$

$$h_2 = \tanh\left(\begin{bmatrix} 0.0788 - 0.02 - 0.1 \\ -0.0197 + 0.05 + 0.3 \end{bmatrix}\right) = \tanh\left(\begin{bmatrix} -0.041 \\ 0.330 \end{bmatrix}\right) = \begin{bmatrix} -0.041 \\ 0.319 \end{bmatrix}$$

Step 3: Classification output

$$o = W_o h_2 = \begin{bmatrix} 0.3(-0.041) - 0.1(0.319) \\ -0.2(-0.041) + 0.4(0.319) \end{bmatrix} = \begin{bmatrix} -0.012 - 0.032 \\ 0.008 + 0.128 \end{bmatrix} = \begin{bmatrix} -0.044 \\ 0.136 \end{bmatrix}$$

Softmax: $P(\text{positive}) = \frac{e^{0.136}}{e^{-0.044} + e^{0.136}} = \frac{1.146}{0.957 + 1.146} = 0.545$

Prediction: **Positive** (correct, since “good movie” is positive).

Worked Example 30: HMM Tagging with Two Tags

Given: Sentence = “a dog runs”

Tags: {Det, Noun, Verb}

Simplified transition matrix $P(t_i|t_{i-1})$:

	Det	Noun	Verb
<start>	0.5	0.3	0.2
Det	0.0	0.9	0.1
Noun	0.0	0.2	0.8
Verb	0.4	0.4	0.2

Emission matrix $P(w|t)$:

	a	dog	runs
Det	0.6	0.0	0.0
Noun	0.0	0.7	0.1
Verb	0.0	0.1	0.8

Viterbi computation:

Word 1 “a”:

$$v_1(\text{Det}) = 0.5 \times 0.6 = 0.30$$

$v_1(\text{Noun}) = 0$, $v_1(\text{Verb}) = 0$ (emission is 0)

Word 2 “dog”:

$$v_2(\text{Noun}) = v_1(\text{Det}) \times P(\text{Noun}|\text{Det}) \times P(\text{dog}|\text{Noun}) = 0.30 \times 0.9 \times 0.7 = 0.189$$

$$v_2(\text{Verb}) = v_1(\text{Det}) \times P(\text{Verb}|\text{Det}) \times P(\text{dog}|\text{Verb}) = 0.30 \times 0.1 \times 0.1 = 0.003$$

Word 3 “runs”:

$$v_3(\text{Verb}) = v_2(\text{Noun}) \times P(\text{Verb}|\text{Noun}) \times P(\text{runs}|\text{Verb}) = 0.189 \times 0.8 \times 0.8 = 0.121$$

$$v_3(\text{Noun}) = v_2(\text{Noun}) \times P(\text{Noun}|\text{Noun}) \times P(\text{runs}|\text{Noun}) = 0.189 \times 0.2 \times 0.1 = 0.004$$

$$v_3(\text{Det}) = v_2(\text{Verb}) \times P(\text{Det}|\text{Verb}) \times P(\text{runs}|\text{Det}) = 0.003 \times 0.4 \times 0 = 0$$

Best path: Det $\rightarrow$ Noun $\rightarrow$ Verb

Tagged sentence: a/Det dog/Noun runs/Verb

Worked Example 31: Porter Stemmer on “formalization”

Word: “formalization”

Step 1a: Does it end in “sses”? No. “ies”? No. “ss”? No. “s”? No (ends in “n”). No change.

Step 1b: Does it end in “eed”? No. “ed”? No. “ing”? No.

Step 1c: Does it end in “y” preceded by consonant? No.

Step 2 ($m > 0$): Check longest matching suffix: - “ational” $\rightarrow$ “ate”: Does not match. - “ization” $\rightarrow$ “ize”: “formalization” ends in “ization”. - Stem before “ization” = “formal”. Measure m of “formal”: f-o-r-m-a-l = CVCVCVC. $m = 3$. - Since $m > 0$, rule applies. - “formalization” $\rightarrow$ “formalize”

Step 3 ($m > 0$): - “icate” $\rightarrow$ “ic”: No. - “ative” $\rightarrow \emptyset$: No. - “alize” $\rightarrow$ “al”: “formalize” ends in “alize”. - Stem before “alize” = “form”. Measure m of “form”: f-o-r-m = CVCVC. $m = 2$. - Since $m > 0$, rule applies. - “formalize” $\rightarrow$ “formal”

Step 4 ($m > 1$): - Check suffixes: “al” $\rightarrow \emptyset$ ($m > 1$). - “formal” ends in “al”. Measure m of “form” = 2. - Since $m = 2 > 1$, remove “al”: “formal” $\rightarrow$ “form”

Step 5a: Does it end in “e” ($m > 1$)? “form” does not end in “e”. No change.

Step 5b: Double “l” ($m > 1$ and *d and *L)? “form” does not end in “l”. No change.

Final stem: “form”

Worked Example 32: CKY Parsing with Ambiguity

Given: Sentence = “old men and women”

Grammar in CNF:

$S \rightarrow NP VP$	$NP \rightarrow Adj NP$
$NP \rightarrow Noun$	$NP \rightarrow NP Conj NP$
$VP \rightarrow Verb NP$	$Adj \rightarrow old$
$Noun \rightarrow men$	$Noun \rightarrow women$
$Conj \rightarrow and$	$Verb \rightarrow saw$

Parse 1: [old [men and women]] — “old” modifies the entire group.

Tree structure:

$NP(Adj(old) NP(Noun(men) Conj(and) Noun(women)))$

Parse 2: [[old men] and [women]] — “old” only modifies “men”.

Tree structure:

$NP(NP(Adj(old) Noun(men)) Conj(and) NP(Noun(women)))$

CKY Table ($T[i, j]$ for words “old men and women”):

Base case (diagonal):

- $T[1, 1] = \{Adj\}$ (“old”)
- $T[2, 2] = \{Noun\}$ (“men”)
- $T[3, 3] = \{Conj\}$ (“and”)
- $T[4, 4] = \{Noun\}$ (“women”)

Length 2 spans:

- $T[1, 2]$: $T[1, 1] = \{Adj\}$, $T[2, 2] = \{Noun\}$
– Rule $NP \rightarrow Adj NP$: matches! $T[1, 2] = \{NP\}$
- $T[2, 3]$: $T[2, 2] = \{Noun\}$, $T[3, 3] = \{Conj\}$ → no rule matches
- $T[3, 4]$: $T[3, 3] = \{Conj\}$, $T[4, 4] = \{Noun\}$ → no rule matches

Length 3 spans:

- $T[1, 3]$:
– Split at $k = 1$: $T[1, 1] = \{Adj\}$, $T[2, 3] = \emptyset$ → no match
– Split at $k = 2$: $T[1, 2] = \{NP\}$, $T[3, 3] = \{Conj\}$ → no rule $NP + Conj$
- $T[2, 4]$:
– Split at $k = 2$: $T[2, 2] = \{Noun\}$, $T[3, 4] = \emptyset$ → no match
– Split at $k = 3$: $T[2, 3] = \emptyset$, $T[4, 4] = \{Noun\}$ → no match

Using a simplified grammar that captures the two readings:

Parse 1 (“old” modifies the whole NP):

$NP \rightarrow Adj NP \rightarrow old NP \rightarrow old (NP Conj NP) \rightarrow old (Noun Conj Noun) \rightarrow old (men and women)$

Parse 2 (“old” modifies only “men”):

$NP \rightarrow NP Conj NP \rightarrow (Adj Noun) Conj Noun \rightarrow (old men) and women$

Both parses are valid in the CKY table, demonstrating **coordination ambiguity**.

Worked Example 33: PCFG Disambiguation

Given: Sentence = “old men and women”

Two possible parse trees with rule probabilities:

Parse Tree A: “old” modifies the entire group [old [men and women]]

- $NP \rightarrow Adj\ NP$ (0.30)
- $Adj \rightarrow old$ (0.80)
- $NP \rightarrow NP\ Conj\ NP$ (0.40)
- $NP \rightarrow Noun$ (0.30)
- $Noun \rightarrow men$ (0.40)
- $Conj \rightarrow and$ (1.00)
- $Noun \rightarrow women$ (0.35)

$$P(\text{Tree A}) = 0.30 \times 0.80 \times 0.40 \times 0.30 \times 0.40 \times 1.00 \times 0.35 = 0.004032$$

Parse Tree B: “old” modifies only “men” [[old men] and women]

- $NP \rightarrow NP\ Conj\ NP$ (0.40)
- $NP \rightarrow Adj\ NP$ (0.30)
- $Adj \rightarrow old$ (0.80)
- $NP \rightarrow Noun$ (0.30)
- $Noun \rightarrow men$ (0.40)
- $Conj \rightarrow and$ (1.00)
- $NP \rightarrow Noun$ (0.30)
- $Noun \rightarrow women$ (0.35)

$$P(\text{Tree B}) = 0.40 \times 0.30 \times 0.80 \times 0.30 \times 0.40 \times 1.00 \times 0.30 \times 0.35 = 0.0012096$$

Result: $P(\text{Tree A}) = 0.004032 > P(\text{Tree B}) = 0.00121$

The PCFG prefers Parse Tree A (“old” modifies the entire group) because the single application of $NP \rightarrow NP\ Conj\ NP$ has higher probability than the nested application in Tree B.

Note: The actual preferred reading depends on the specific probabilities assigned by the grammar trained on a treebank.

Worked Example 34: Dependency Parsing – Non-Projective Case

Given: Sentence = “John saw Mary yesterday”

Gold dependency tree:

- saw → John (nsubj)
- saw → Mary (obj)
- yesterday → saw (advmod)

Projectivity check: For “John saw Mary yesterday”, the arc from yesterday (pos 4) to saw (pos 2) spans Mary (pos 3). Since Mary is directly dependent on saw, this arc is **projective**. For a genuine non-projective example, consider “John saw yesterday Mary” (ungrammatical in English but possible in some languages):

Non-projective tree:

- saw → John (nsubj)
- saw → Mary (obj)
- yesterday → saw (advmod)

In the sentence “John saw yesterday Mary”, the arc from yesterday (pos 3) to saw (pos 2) spans no other words (adjacent). The arc from Mary (pos 4) to saw (pos 2) spans yesterday (pos 3). Is yesterday a descendant of saw? No, yesterday is a sibling dependent. So this arc is **non-projective**.

Shift-reduce parsers that enforce projectivity cannot produce this tree. Algorithms like the arc-eager or arc-standard variants with pseudo-projective transformations are needed.

6 Module 6: Discourse Analysis

6.1 Introduction to Discourse Analysis

Discourse analysis is the study of how sentences in a text or conversation relate to one another to form a coherent whole. While syntax analyzes the structure of individual sentences and semantics studies the meaning of words and sentences, discourse analysis examines how multiple sentences work together to convey meaning beyond what any single sentence expresses.

The two primary tasks in discourse analysis are:

1. **Coreference resolution:** Identifying which linguistic expressions (pronouns, definite noun phrases, names) refer to the same discourse entity.
2. **Coherence relation identification:** Determining the logical and semantic relationships between sentences (e.g., cause-effect, contrast, elaboration).

6.2 Coreference Resolution

Coreference resolution is the task of determining whether two or more referring expressions in a text refer to the same discourse entity (the same person, object, or concept). When two expressions refer to the same entity, they are said to **corefer**.

Example: Coreference Chain

“Victoria Chen, CFO of Megabucks Banking, saw her pay jump to \$2.3 million, as the 38-year-old became the company’s president. It is widely known that she came to Megabucks from rival Lotsabucks.”

Coreference chains:

1. {Victoria Chen, her, the 38-year-old, she}
2. {Megabucks Banking, the company, Megabucks}
3. {her pay}
4. {Lotsabucks}

6.2.1 The Discourse Model

A **discourse model** is a mental representation built incrementally while understanding a text or conversation. It contains:

- **Entities:** People, objects, and ideas mentioned in the discourse.
- **Properties:** Features and attributes of these entities.
- **Relationships:** How entities relate to one another.

When an entity is first mentioned, a new representation is **evoked** (created) in the model. Subsequent mentions **access** the existing representation rather than creating a new one. This incremental construction allows listeners and readers to track who and what is being discussed.

6.2.2 References and Anaphora

A **reference** is a linguistic expression used to denote an entity. When a referring expression points to an entity previously introduced into the discourse, this is called **anaphora**, and the expression is called an **anaphor** or **anaphoric** expression.

Key terminology:

- **Mention / Referring expression:** The linguistic expression used (e.g., “Victoria Chen”, “her”, “she”).
- **Referent:** The actual entity being referred to (e.g., the person Victoria Chen).
- **Antecedent:** The prior mention that licenses the use of an anaphoric expression. For example, “Victoria Chen” is the antecedent of “she”.
- **Singleton:** An entity mentioned only once in a text (no anaphoric references).

6.2.3 Anaphora vs. Coreference

While the terms are related, **not all anaphoric relations are coreferential**. Understanding this distinction is crucial for discourse analysis.

- **Anaphora:** A referring expression points back (or forward) to another expression in the text. The relationship is about **interpretation dependency**.
- **Coreference:** Two expressions refer to the **same discourse entity**. The relationship is about **identity of reference**.

Key Distinction: Anaphoric but Not Coreferential

Example: “We went to see a **movie** last night. The **tickets** were expensive.”

- “The tickets” is **anaphoric** because its interpretation depends on the prior mention of going to see a movie.
- However, “movie” and “tickets” are **NOT coreferential** — they refer to different entities (the film vs. the admission passes).

This shows that anaphora is a broader concept than coreference. Coreference chains capture identity; anaphoric relations capture interpretive dependency.

6.2.4 Cataphora

While **anaphora** involves backward reference to an antecedent, **cataphora** involves **forward reference** — the referring expression appears **before** the entity it refers to.

Example: Cataphora

Example: “**Despite her difficulty**, Swathi went ahead to help him.”

- “her” appears before “Swathi” is mentioned.
- The referent follows the referring expression, making this a **cataphoric** reference.
- Cataphora is less common than anaphora but occurs in certain syntactic constructions.

6.2.5 Types of Referring Expressions

Referring expressions are classified into six types:

1. **Indefinite Noun Phrases:** Introduce new entities into the discourse.
 - Example: “Ram had gone to bring **him some food**” — “some food” is new to the hearer.
2. **Definite Noun Phrases:** Refer to entities already known or identifiable.

- Example: “I used to read **The Times of India**” — the newspaper is a specific, known entity.
3. **Pronouns**: A form of definite reference using pronouns like “he”, “she”, “it”, “they”.
 - Example: “Ram laughed as loud as **he** could.”
 4. **Demonstratives**: “this” and “that” behave differently from simple pronouns, often requiring proximity or salience.
 - Example: “**This** is what I meant.”
 5. **Names**: The simplest type — proper nouns for people, organizations, locations.
 - Example: “**Ram** is a manager.”
 6. **One-anaphora**: The use of “one” can be paraphrased as “one of them.”
 - Example: “John has two girlfriends, but I only have **one**.”

6.2.6 Types of Referents

Some referents complicate coreference resolution:

- **Inferables**: Refer to entities not explicitly mentioned but inferentially related to an evoked entity.
 - Example: “I almost bought an Acura Integra today, but a **door** had a dent and the **engine** seemed noisy.” The door and engine are inferably parts of the Integra.
- **Discontinuous sets**: Plural referring expressions like “they” and “them” refer to sets of entities evoked together.
 - Example: “John and Mary love their Acuras. **They** drive **them** all the time.” “They” = {John, Mary}, “them” = {their Acuras}.
- **Generics**: Refer to a class in general, not specific instances.
 - Example: “I saw no less than 6 Acura Integras today. **They** are the coolest cars.” “They” refers to the class of Integras, not the 6 specific ones.

6.2.7 Coreference Resolution Tasks

Coreference resolution comprises two main tasks:

1. **Coreference resolution (full)**: Finds all words and phrases that refer to the same entity and links them into **coreference chains**.
 - Challenge: The pronoun “it” is tricky — sometimes it has no real referent (e.g., “It’s raining”).
2. **Pronominal anaphora resolution**: Focuses on one pronoun at a time and finds its antecedent.
 - Example: “Ram lost **his** bag” — “his” refers to Ram.

6.3 Constraints and Preferences in Coreference Resolution

Resolving references requires both hard constraints (which must be satisfied) and soft preferences (which guide selection among valid candidates).

6.3.1 Syntactic and Semantic Constraints

1. **Number agreement:** Referring expressions and their referents must agree in number.
 - “The **cats** slept. **They** were tired.” (plural “they” matches plural “cats”)
2. **Person agreement:** Pronouns must match the person of their antecedent (first, second, third).
3. **Gender agreement:** Referents must match the gender specified by the pronoun (male “he”, female “she”, non-personal “it”).
4. **Binding theory:** Reflexive pronouns (“himself”, “herself”, “themselves”) must refer to the subject of the most immediate clause, while non-reflexives cannot.
 - “John bought **himself** a new Acura.” [himself = John]
 - “John bought **him** a new Acura.” [him $\neq$ John]
5. **Selectional restrictions:** Verbs place constraints on their arguments that eliminate impossible referents.
 - “John parked his Acura in the garage. He had driven **it** around for hours.” “it” must refer to “Acura” (drivable), not “garage” (not drivable).

6.3.2 Preferences in Pronoun Interpretation

When multiple candidates satisfy constraints, preferences guide resolution:

1. **Recency:** Recently mentioned entities are more likely to be referred to.
 - “John has an Integra. Bill has a Legend. Mary likes to drive **it**.” “it” is more likely to refer to “Legend” (most recent) than “Integra”.
2. **Grammatical role:** Subjects are more salient than objects; pronouns often refer to the subject of the previous sentence.
 - “**John** went to the Acura dealership with Bill. **He** bought an Integra.” “He” = John (subject).
3. **Repeated mention:** Entities mentioned multiple times become more salient and are more likely to be referred to again.
4. **Parallelism:** Pronouns tend to match the grammatical position of their antecedent.
 - “Mary went with Sue to the Acura dealership. Sally went with **her** to the Mazda dealership.” “her” = Sue (parallel object position).
5. **Verb semantics:** The meaning of the verb affects which noun is more important.
 - “John **telephoned** Bill. He lost the pamphlet.” “He” = John (subject-focused verb).
 - “John **criticized** Bill. He lost the pamphlet.” “He” = Bill (object-focused verb).

6.3.3 Hobbs Algorithm

The **Hobbs algorithm** is a classic algorithm for pronoun resolution that uses syntactic parse trees to locate the correct antecedent. The algorithm combines constraints and preferences systematically.

Key observations:

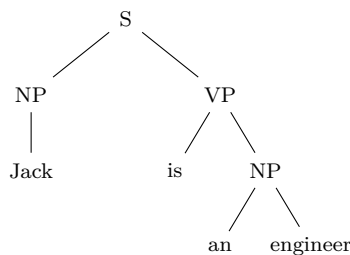
- Search is always done to the left of the pronoun.

- Closer nouns are preferred (recency).
- Gender and number must match.
- Syntactic constraints (binding theory) eliminate impossible candidates.

Hobbs Algorithm steps:

1. Begin at the NP node immediately dominating the pronoun.
2. Go up the tree to the first NP or S node encountered. Call this node X, and call the path used to reach it p.
3. Traverse all branches below node X to the left of path p in a left-to-right, breadth-first fashion. Propose as antecedent any NP node encountered that has an NP or S node between it and X.
4. If node X is the highest S node in the sentence, traverse the parse trees of previous sentences in order of recency (most recent first), each traversed left-to-right, breadth-first. Propose any NP encountered as antecedent.
5. From node X, go up the tree to the first NP or S node encountered. Call this new node X, and call the path traversed to reach it p.
6. If X is an NP node and the path p to X did not pass through the Nominal node that X immediately dominates, propose X as the antecedent.
7. Traverse all branches below node X to the left of path p in a left-to-right, breadth-first manner. Propose any NP node encountered as the antecedent.
8. If X is an S node, traverse all branches of node X to the right of path p in a left-to-right, breadth-first manner, but do not go below any NP or S node encountered. Propose any NP node encountered as the antecedent.
9. Go to step 4.

Sentence 1: “Jack is an engineer.”



Sentence 2: “Jill likes him.”

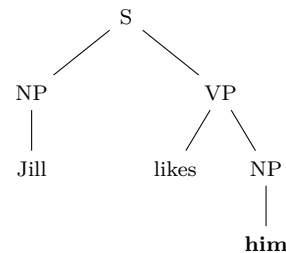


Figure 19: Parse trees for the Hobbs algorithm example. The algorithm searches from “him” (Sentence 2) leftward across parse trees.

Worked Example: Hobbs Algorithm Application

Sentence 1: “Jack is an engineer.” **Sentence 2:** “Jill likes **him**.”

Step-by-step resolution:

1. Start from the pronoun “him” in Sentence 2.
2. Move up to the first NP or S node dominating “him”.
3. Search left branches within the same sentence. The only NP to the left is “Jill”.
4. **Gender check:** “Jill” is female, “him” is male — mismatch! Reject Jill.
5. Move to the previous sentence (Sentence 1).
6. Search left-to-right, breadth-first in Sentence 1.
7. First NP encountered: “Jack” (subject position).
8. Second NP encountered: “engineer” (object position).
9. **Grammatical role preference:** Subject “Jack” is preferred over object “engineer”.
10. **Result:** “him” refers to “Jack”.

6.4 Coherence Relations

Coherence relations are the logical and semantic relationships that hold between adjacent sentences or clauses in a discourse, making the text coherent and meaningful. Identifying these relations is essential for understanding how ideas connect.

Common coherence relations include:

- **Cause-Effect:** One event causes another. (“The roads were flooded, **so** the match was postponed.”)
- **Contrast:** Unexpected or opposing outcomes. (“She studied hard. **However**, she did not perform well.”)
- **Elaboration:** Additional detail or explanation. (“He bought a car. **Specifically**, a red Honda.”)
- **Sequence:** Temporal ordering of events. (“He collected the data, **then** analyzed the results.”)
- **Condition:** One event depends on another. (“**If** it rains, the event will be canceled.”)

Worked Example: Identifying Coherence Relations

Given sentences: “She did not set an alarm. She woke up late.”

Step 1: Identify the implicit logical connection.

- Not setting an alarm **caused** waking up late.

Step 2: Classify the relation.

- **Cause-Effect:** Cause = “She did not set an alarm.” Effect = “She woke up late.”

Step 3: Rewrite with an explicit connective.

- “She did not set an alarm, **so** she woke up late.”
- “**Because** she did not set an alarm, she woke up late.”

6.5 Discourse Structure Parsing

Discourse structure parsing is the task of analyzing a text into discourse units and representing the coherence relations between them, typically as a tree or dependency structure.

A **discourse unit** is a minimal clause or sentence that expresses a complete proposition. Relations between units are labeled with coherence relation types.

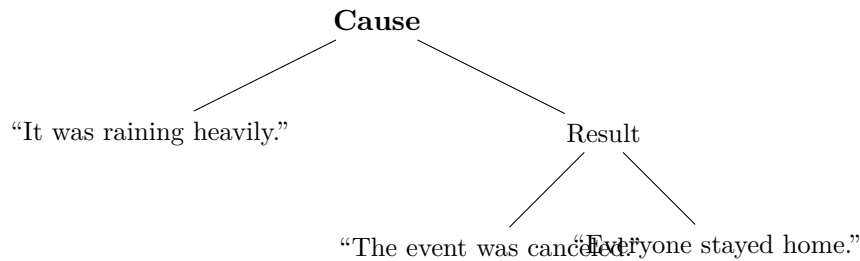


Figure 20: Discourse structure tree for “It was raining heavily. Therefore, the event was canceled. Everyone stayed home.”

Worked Example: Discourse Structure Parsing

Given: “He missed the bus, so he was late for work.”

Step 1: Identify discourse units.

- Unit 1: “He missed the bus.”
- Unit 2: “he was late for work.”

Step 2: Identify discourse marker.

- “so” indicates a result/effect relation.

Step 3: Assign relation.

- Relation: **Cause-Effect**
- Cause: Unit 1 (missed the bus)
- Effect: Unit 2 (late for work)

Parsed structure:

[Cause] He missed the bus $\xrightarrow{\text{so}}$ [Effect] he was late for work

6.6 Global Coherence

Global coherence evaluates whether an entire text or paragraph is coherent as a whole, not just at the level of individual sentence pairs. A globally coherent text maintains a consistent topic, logical flow, and appropriate use of transitions.

Indicators of global coherence:

1. **Topic consistency:** All sentences relate to a central theme.
2. **Logical ordering:** Ideas are presented in a sensible sequence (e.g., chronological, cause-to-effect).
3. **Appropriate transitions:** Connectives and transitional phrases guide the reader through the argument.

Worked Example: Evaluating Global Coherence

Given paragraph:

“She loves painting. The stock market crashed yesterday. Her favorite color is blue.”

Evaluation: Not globally coherent.

Issues:

1. Sentence 1: Topic = painting/art.
2. Sentence 2: Topic = stock market (unrelated to painting).
3. Sentence 3: Topic = color preference (weakly related to painting).

Revised (coherent) version:

“She loves painting. Her favorite color is blue, which she often uses in her artwork.”

Worked Example: Enhancing Global Coherence

Given: “The project started late. It was completed successfully. The team celebrated.”

Step 1: Identify the logical sequence.

- Late start → successful completion → celebration

Step 2: Add appropriate transitions to strengthen coherence.

Improved version:

“**Although** the project started late, it was completed successfully. **Therefore**, the team celebrated.”

Transitions used:

- “Although” signals contrast (late start vs. success).
- “Therefore” signals result (success leads to celebration).

7 Summary of Key Concepts by Module

7.1 Module 1: N-gram Language Models and Edit Distance

- **Language models** assign probabilities to word sequences and predict upcoming words.
- **N-grams** model sequences of n words; bigrams and trigrams are most common.
- **MLE** estimates probabilities from training corpus counts: $P(w_n \mid w_{n-1}) = C(w_{n-1}, w_n) / C(w_{n-1})$.
- **Perplexity** measures model quality: $PP(W) = P(W)^{-1/N}$. Lower is better.
- **Smoothing** (Laplace, add-k, interpolation, stupid backoff) handles unseen n-grams.
- **Entropy** and **cross-entropy** provide information-theoretic foundations for perplexity.
- **Edit distance** measures string similarity via minimum insertion, deletion, and substitution operations.

7.2 Module 2: Word Senses, WordNet, and Vector Semantics

- **Word senses** are discrete meaning representations; polysemy and homonymy describe multiple senses.
- **Lexical relations**: synonymy, antonymy, hyponymy, hypernymy, meronymy.
- **WordNet** is a large lexical database organizing words into synsets with glosses and sense relations.
- **WSD** selects the correct sense in context; best modern approach uses contextual embeddings (BERT).
- **PMI/PPMI** measures word co-occurrence association: $\text{PMI}(w, c) = \log_2 \frac{P(w, c)}{P(w)P(c)}$.
- **Word Sense Induction** discovers senses automatically via clustering of context vectors.

7.3 Module 3: Naive Bayes and Text Classification

- **Text classification** assigns labels to documents; applications include sentiment analysis and spam detection.
- **Naive Bayes** is a generative classifier using Bayes' rule with conditional independence and bag-of-words assumptions.
- **Training**: Estimate $P(c)$ and $P(w | c)$ with MLE + Laplace (add-1) smoothing.
- **Optimizations**: Binary multinomial NB, negation handling, sentiment lexicons.

7.4 Module 4: Deep Learning for Sequences

- **RNNs** process sequences via hidden state recurrence: $h_t = \tanh(W_h h_{t-1} + W_x x_t + b_h)$.
- **BPTT** trains RNNs by unrolling through time and backpropagating gradients.
- **Vanishing/exploding gradients** limit RNNs' ability to learn long-range dependencies.
- **LSTM** solves this via cell state and forget/input/output gates.
- **GRU** simplifies LSTM with reset and update gates, using only the hidden state.
- **Bi-RNNs** process sequences in both directions, using full context but doubling computation.

7.5 Module 5: Morphology and Syntax

- **Morphology** studies word structure; morphemes are the smallest meaning-bearing units (free vs. bound, lexical vs. grammatical, inflectional vs. derivational).
- **Regular expressions** use disjunction, Kleene operators, anchors, and capture groups for pattern matching.
- **FSA/DFA** recognize patterns via state transitions; 5-tuple formalism.
- **Porter Stemmer** applies 5 sequential steps with measure-based conditions to strip suffixes.
- **POS tagging** assigns grammatical categories; approaches include rule-based, HMM, and transformation-based.
- **HMM** uses transition and emission probabilities; Viterbi algorithm finds the optimal tag sequence.
- **CFG** defines syntactic structure via production rules with terminals and non-terminals.

- **CKY** parses CFGs in CNF using dynamic programming ($O(N^3)$).
- **PCFG** adds probabilities to CFG rules for disambiguation.
- **Dependency parsing** uses shift-reduce with LEFTARC/RIGHTARC/SHIFT to build head-dependent trees.

7.6 Module 6: Discourse Analysis

- **Coreference resolution** identifies expressions referring to the same discourse entity; builds coreference chains.
- **Discourse model** incrementally tracks entities, properties, and relationships across a text.
- **Anaphora**: anaphoric expressions refer back to previously introduced antecedents.
- **Referring expressions**: indefinite/definite NPs, pronouns, demonstratives, names, one-anaphora.
- **Constraints**: number, person, gender, binding theory, selectional restrictions.
- **Preferences**: recency, grammatical role, repeated mention, parallelism, verb semantics.
- **Hobbs algorithm** uses parse tree traversal (left-to-right, breadth-first) to find antecedents.
- **Coherence relations**: cause-effect, contrast, elaboration, sequence, condition.
- **Discourse structure parsing**: analyzes text into discourse units with labeled relations.
- **Global coherence**: topic consistency, logical ordering, appropriate transitions.